Homomorphic Field Trace Revisited : Breaking the Cubic Noise Barrier

Kang Hoon Lee and Ji Won Yoon

Korea University, School of CyberSecurity
{hoot55,jiwon_yoon}@korea.ac.kr

Abstract. We present a novel homomorphic trace evaluation algorithm RevHomTrace, which mitigates the phase amplification problem that comes with the definition of the field trace. Our RevHomTrace overcomes the phase amplification with only a negligible computational overhead, thereby improving the usability of the homomorphic field trace algorithm. Moreover, our tweak also improves the noise propagation of the HomTrace and breaks the traditional $\mathcal{O}\left(N^3\right)$ variance bound in previous works into $\mathcal{O}(N \log N)$.
Our experimental results obtained by integrating RevHomTrace into state-of-the-art homomorphic encryption algorithms further demonstrate the usefulness of our algorithm. Specifically, RevHomTrace improves the noise accumulation of the (high precision) circuit bootstrapping, which also achieves maximal 1.30× speedup by replacing the costly high precision trace evaluation. Also, based on our idea of RevHomTrace, we present a low latency, high precision LWE-to-GLWE packing algorithm `MS-PackLWEs`. We also show that our `MS-PackLWEs` significantly reduces the packing error without severe degradation of performance.

Keywords: Homomorphic Encryption · Field Trace · FHEW · TFHE

1 Introduction

Fully homomorphic encryption (FHE) scheme is an encryption scheme that supports the computation of a circuit of arbitrary depth over encrypted data without decryption. Due to this functionality, it has emerged as a solution to outsource their data, but keep their data hidden. But due to the inefficiency of FHE, many investigations have been carried out to improve its efficiency after the first blueprint by Gentry [Gen09].

Well-known FHE schemes such as FHEW/TFHE [DM15, CGGI20], BFV/BGV [FV12, Bra12, BGV14], and CKKS [CKKS17] mostly rely on the hardness of the Learning with Errors (LWE) problem [Reg09] and its ring variant, Ring Learning with Errors (RLWE) [LPR10]. Due to this foundation, a fundamental characteristic inherent to (R)LWE-based encryption schemes is the presence of noise in ciphertexts. However, this noise presents a significant computational challenge as it grows during homomorphic operations such as addition, multiplication, keyswitching, etc. This noise accumulation limits the depth of circuits that can be evaluated homomorphically, making noise level control inevitable for ensuring correctness and computational feasibility.

To address the noise accumulation problem, the so-called 'bootstrapping' technique was developed that effectively refreshes ciphertexts by homomorphically evaluating the decryption circuit. After bootstrapping, the noise inside the ciphertext is reduced and can be used for further computation. However, bootstrapping itself is computationally

expensive procedure and inhibits the overall usage of homomorphic encryption. Thus, beyond developing efficient bootstrapping procedures, minimizing the noise growth of fundamental homomorphic algorithms remains crucial for overall system performance, as reduced noise growth directly translates to fewer bootstrapping operations and improved computational efficiency.

One of the obstacles in this area is the homomorphic trace evaluation (HomTrace) [CDKS21], recently used in various fields of homomorphic encryption, including the faster circuit bootstrapping [WWL+24, WHS+24, LSL+25] in the leveled homomorphic encryption (LHE) mode of TFHE [CGGI17]. Moreover, it has also been utilized for packing multiple LWEs into General-LWE (GLWE) [Zam25] ciphertext [CDKS21], which was used for scheme switching from TFHE to BFV [LSL+25] and many other applications [KDE+23, MW24, ADDG24, HYT+24, BZS+24]. Last but not least, the HomTrace was also deployed for 'unrolling', i.e., decompressing the compressed bootstrapping key for FHEW-like schemes [KLD+23].

Nonetheless, as reported in these HomTrace-based schemes, the HomTrace algorithm suffers from two obstacles: **(1)** Phase Amplification and **(2)** Noise Accumulation. In detail, the message amplified by the factor of N, the GLWE ring dimension. Also, the HomTrace adds an additional noise with variance $\mathcal{O}(N^3)$. Both challenges come from the structure of the HomTrace algorithm of [CDKS21], and several approaches [WWL+24, WHS+24] have been made to overcome this challenge, but none of these approaches include modifying the HomTrace algorithm itself.

1.1 Related Works

From the best of our knowledge, the homomorphic evaluation of the field trace was first utilized to switch the field of the RLWE ciphertext into a smaller field [GHPS13, GHS12]. On the other hand, it was until the work of [CDKS21] where they introduced the HomTrace that works on the underlying plaintext of the ciphertext that switches an LWE ciphertext into a GLWE ciphertext. Based on this basic algorithm, they also presented an algorithm that packs multiple LWE ciphertexts into a single RLWE ciphertext (LWE(s)-to-RLWE). While their packing algorithm favors in terms of computational complexity compared to previous packing keyswitching techniques [CGGI17, MS18], they suffer from rather severe noise growth of $\mathcal{O}(N^3)$.

After the work of [CDKS21], in a possibly orthogonal direction, but quite similar in terms of 'zeroizing' out the unwanted coefficients, the HomTrace was later used for circuit bootstrapping [WWL+24] to replace the heavy *private keyswitching* process [CGGI17]. Through this replacement, they achieved about 10× speedup for circuit bootstrapping compared to the original circuit bootstrapping of [CGGI17]. While they also suffer from the factor N that is multiplied to the phase of the ciphertext, they drifted this problem by multiplying the $N^{-1} \pmod q$ on the message, thereby canceling out the factor N after HomTrace. Nonetheless, as the N^{-1} is only multiplied on the message, the noise in their ciphertext is still amplified by N. Later on, this problem was pointed out by [WHS+24], and they tried to handle this issue in two different domains: NTT and FFT domain. In the NTT domain, where N^{-1} exists, they multiplied the N^{-1} to both the message and the error term making use of the pre-processing of [CDKS21]. In the FFT domain, where the inverse of N does not exist, they employed modulus switching and modulus raising to cancel out the phase amplification. With this improvement, they achieved 3.5× speedup compared to the previous work of [WWL+24].

Table 1: Comparison between existing phase amplification removal HomTrace evaluation methods with our RevHomTrace. The k and the k' denotes the GLWE rank, where $k \leq k'$. Also, q denotes the ciphertext modulus.

Method	Complexity	Noise Variance	Key Size
[CDKS21] $\times N^{-1}$	$\mathcal{O}(k^2 N \log^2 N)$	$\mathcal{O}(N^3)$	$\mathcal{O}(kN \log q)$
[WHS+24] Modulus Switch	$\mathcal{O}(k^2 N \log^2 N)$	$\mathcal{O}(N^3)$	$\mathcal{O}(kN \log q)$
[WHS+24] Rank Switch	$\mathcal{O}(k'^2 N \log^2 N)$	$\mathcal{O}(N^3)$	$\mathcal{O}(k'N \log q)$
Ours	$\mathcal{O}(k^2 N \log^2 N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(kN \log q)$

1.2 Our Contributions

In this paper, we address the aforementioned phase amplification problem and further improve the noise accumulation of the HomTrace evaluation from the prior work of [WHS+24]. Thus, we propose an amplification removal HomTrace evaluation algorithm called RevHomTrace. Our RevHomTrace adds a simple tweak from the preprocessing HomTrace of [WWL+24], by applying the modulus switching technique on each iteration, and by evaluating the automorphisms in reverse order.

From these simple tweaks, we show that the RevHomTrace naturally removes the factor N from the phase with only a negligible computational overhead that comes from the modulus switching. Moreover, our algorithm only adds noise with variance of only $\mathcal{O}(N \log N)$, which is comparable to the traditional $\mathcal{O}(N^3)$ bound known in the literature. We provide a detailed noise analysis and compare it with the existing solutions that aim to fix the amplification after the HomTrace. A brief comparison between phase amplification removal HomTrace algorithms including our RevHomTrace is depicted in Table 1.

Next, using our novel RevHomTrace, we show our RevHomTrace can be generally applied to existing applications [CDKS21, WWL+24, WHS+24, LSL+25] that used HomTrace as a subroutine, and show the improvements with our RevHomTrace. The detailed applications are listed as follows:

A. High Precision Circuit Bootstrapping. During the circuit bootstrapping of [WWL+24, WHS+24], they employ the HomTrace to 'zeroize' the non-constant coefficients of the message from the GLWE ciphertext. Our RevHomTrace can adapted to these scenarios, and help reduce the overall noise level. Our experimental results achieves 1.30 times of speedup from the work of [WHS+24] with increased computational depth for the high precision circuit bootstrapping mode. Moreover, with the tweak of parameters, we also show that with similar runtime, we can considerably increase the computation depth of the circuit-bootstrapped ciphertext.

B. Ring Packing. We also apply our tweak to the LWEs-to-GLWE ring packing of [CDKS21]. With our tweak, we mitigate the noise accumulation from $\mathcal{O}(N^3)$ to $\mathcal{O}(N \log N)$. We experimentally show in Table 6 that with the tradeoff of approximately 10% increase of latency, we can reduce the average noise by 8 bits compared to the original ring packing of [CDKS21]. Also, our comparison with the traditional

ring packing from [CGGI20, MS18] shows that our packing method shows similar or better noise accumulation, but with latency smaller than an order of magnitude.

2 Backgrounds

2.1 Preliminaries

Throughout the paper, let N be a power of two and denote the $2N$-th cyclotomic ring as $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_{q,N} = \mathcal{R}/q\mathcal{R}$ for $q \in \mathbb{Z}$. Also, bold case letters indicate vectors of matrices unless mentioned otherwise. We denote $[\cdot]_q$ the modulo q reduction, and for a positive integer $n \in \mathbb{N}$, we denote $[n] = \{1, 2, \cdots, n\}$, and for two integers $a, b \in \mathbb{Z}$ with $a \leq b$, $[a, b]$ denote the set $\{a, a+1, \cdots, b\}$, and $[\![a, b[\![$ denote the set $\{a, a+1, \cdots, b-1\}$. If not mentioned otherwise, all logarithms are base 2 throughout the paper.

2.2 FHEW-Like Cryptosystems

In this section, we first review the FHEW-like HE systems starting from their various variants of ciphertexts along with the core algorithms used in these schemes. Throughout the paper, we basically follow the notations of [Zam25], providing detailed explanations towards the TFHE scheme. We highly recommend the readers to refer to the document if necessary.

2.2.1 LWE, RLWE, and GLWE Ciphertexts.

Given a message $M \in \mathcal{R}_{p,N}$ and a secret key $\mathbf{S} = (S_1, S_2, \cdots, S_k) \in \mathcal{R}_{q,N}^k$, a GLWE encryption $\mathbf{C}$ of M under $\mathbf{S}$ is denoted as:

$$\mathbf{C} = \left(A_1, \cdots, A_k, B = \sum_{i=1}^{k} A_i \cdot S_i + [M \cdot \Delta]_q + E \right) \in \mathsf{GLWE}_{q,\mathbf{S}}(M) \subseteq \mathcal{R}_{q,N}^{k+1},$$

where each $A_i \in \mathcal{R}_{q,N}$ is a polynomial whose coefficients are uniformly sampled from $\mathbb{Z}_q$, and E is a noise polynomial whose coefficients are sampled from a Gaussian error distribution χ_σ for some σ. Also, the $\Delta \leq \lfloor q/p \rceil$ is called a scaling factor.

For a special case of GLWE where $N = 1$ and $k = n$ for some $n \in \mathbb{N}$, the GLWE ciphertext is then called a LWE ciphertext, where it is denoted as

$$(a_1, \cdots, a_n, b) \in \mathbb{Z}^{n+1}.$$

On the other hand, when we set $N > 1$ and $k = 1$, then the GLWE ciphertext is called an RLWE ciphertext, formulated as two polynomials $(A, B) \in \mathcal{R}_{q,N}^2$.

The decryption of a GLWE ciphertext first starts with calculating its *phase*,

$$\bar{M} = B - \sum_{i=1}^{k} A_i \cdot S_i = M \cdot \Delta + E \in \mathcal{R}_{q,N}.$$

After we retrieve its phase, we perform division by its scaling factor Δ, and round it to its closest element in $\mathcal{R}_{p,N}$.

2.2.2 Gadget Decomposition.

A general definition for the *gadget toolkit* is that given a modulus q and a positive integer ℓ, there exists a fixed gadget vector $\mathbf{g} = (g_1, \cdots, g_\ell) \in \mathcal{R}_{q,N}^\ell$, and gadget decomposition function $h : \mathcal{R}_{q,N} \to \mathcal{R}^\ell$, they satisfy $\langle h(\mathbf{a}), \mathbf{g} \rangle = \mathbf{a} \in \mathcal{R}_{q,N}$ and the coefficients of $h(\mathbf{a})$ are bounded by some boundary B. In FHEW-like cryptosystems [DM15, CGGI20], they mostly use the *base decomposition*, where the gadget vector is defined as $\mathbf{g} = \left(\left\lceil \frac{q}{B} \right\rfloor, \left\lceil \frac{q}{B^2} \right\rfloor, \cdots, \left\lceil \frac{q}{B^\ell} \right\rfloor\right)$ for $B^\ell < q$ for some positive integer base $B \in \mathbb{Z}$. Then the corresponding decomposition $h(\mathbf{a}) \in \mathcal{R}_B^\ell$ for $\mathbf{a} \in \mathcal{R}_{q,N}$ is given as vectors of polynomials $(a_1, \cdots, a_\ell) \in \mathcal{R}_B^\ell$ that satisfy

$$\mathbf{a} = \sum_{i=1}^{\ell} a_i \cdot \left\lceil \frac{q}{B^i} \right\rfloor + E_{\mathsf{Dec}},$$

where the coefficients $a_{i,j}$ of a_i are within the bound $-\frac{B}{2} \leq a_{i,j} < \frac{B}{2}$ for all $i \in [\ell]$. Also, $E_{\mathsf{Dec}} \in \mathcal{R}_{q,N}$ denotes the decomposition error, whose coefficients $E_{\mathsf{Dec},i}$ satisfy $|E_{\mathsf{Dec},i}| \leq \left\lceil \frac{q}{2 \cdot B^\ell} \right\rfloor$

2.2.3 RLev and GLev Ciphertexts.

Using the aforementioned gadget decomposition, we define GLev ciphertext which is usually utilized to make keyswitching key in FHEW-like cryptosystems, as we can perform homomorphic multiplications between an element in $\mathcal{R}_{q,N}$ and GLev ciphertext. Detailed key switching procedure and its noise formula is depicted in Appendix B.2.

Given a decomposition base $B \in \mathbb{N}$ and a decomposition level $\ell \in \mathbb{N}$, a GLev ciphertext $\overline{C} \in \mathcal{R}_{q,N}^{\ell \times (k+1)}$ of a plaintext $M \in \mathcal{R}_{q,N}$ under the GLWE secret $\mathbf{S} \in \mathcal{R}_{q,N}^k$ is a length ℓ vector of GLWE ciphertext:

$$\overline{C} = \left(\mathsf{GLWE}_{q,\mathbf{S}}\left(\left\lceil \frac{q}{B^i} \right\rfloor \cdot M\right)\right)_{i \in [\ell]} \in \mathsf{GLev}_{q,\mathbf{S}}^{B,\ell}(M)$$

Likewise, a special case for GLev ciphertext with $k = 1$ and $N > 1$ is called the RLev ciphertext.

2.2.4 Gadget Product.

For $u \in \mathcal{R}_{q,N}$, the gadget product $\odot : \mathcal{R}_{q,N} \times \mathsf{GLev}_{q,\mathbf{S}}^{B,\ell} \to \mathsf{GLWE}_{q,\mathbf{S}}$ is defined as

$$\begin{aligned} u \odot \mathsf{GLev}_{q,\mathbf{S}}(M) &= \left\langle h(u), \left(\mathsf{GLWE}_{q,\mathbf{S}}\left(\left\lceil \frac{q}{B^i} \right\rfloor \cdot M\right)\right)_{i \in [\ell]} \right\rangle \\ &= \mathsf{GLWE}_{q,\mathbf{S}}\left(\langle h(u), \mathbf{g} \rangle \cdot M\right) \\ &= \mathsf{GLWE}_{q,\mathbf{S}}\left(u \cdot M\right). \end{aligned}$$

From the definition, the complexity of a single gadget product is equal to $(k+1) \cdot \ell$ polynomial multiplications. The error after the gadget product is $\sum_{i=1}^{\ell} u_i \cdot e_i$, where u_i is each polynomial element of the decomposition $h(u)$, and e_i is the error of each GLWE ciphertext of the GLev ciphertext. Thanks to the decomposition, we can bound the error up to some level by adjusting the base B and the length ℓ.

2.2.5 GGSW Ciphertext.

The GGSW ciphertext is capable of homomorphic multiplication between GLWE and GGSW ciphertext. Given a decomposition base B and decomposition length ℓ, a GGSW ciphertext $\overline{\overline{\mathbf{C}}} \in \mathcal{R}_{q,N}^{(k+1)\ell\times(k+1)}$ of a plaintext $M \in \mathcal{R}_{q,N}$ under the GLWE key $\mathbf{S} \in \mathcal{R}_{q,N}^{k}$ is a length $(k+1)$ vector of GLev ciphertexts:

$$\overline{\overline{\mathbf{C}}} = \left(\mathsf{GLev}_{q,\mathbf{S}}^{B,\ell}(-S_1 \cdot M), \cdots, \mathsf{GLev}_{q,\mathbf{S}}^{B,\ell}(-S_{k+1} \cdot M)\right) \in \mathsf{GGSW}_{q,\mathbf{S}}^{B,\ell}(M)$$

where $S_{k+1} = -1$. Likewise, a GGSW ciphertext with $k = 1$ and $N > 1$ is called an RGSW ciphertext.

2.3 Bootstrapping in FHEW-Like Cryptosystems

We now introduce two different bootstrapping methods deployed in FHEW-like cryptosystems: **(1) Programmable Bootstrapping**, and **(2) Circuit Bootstrapping**.

2.3.1 Programmable Bootstrapping.

Given an LWE ciphertext $c \in \mathsf{LWE}_s(m)$ and a negacyclic function $f : \mathbb{Z}_{2t} \to \mathbb{Z}_{2t}$ with $f(i+t) = -f(i)$, the programmable bootstrapping in FHEW-like cryptosystems homomorphically evaluates f on m while refreshing the ciphertext for further computation, returning $c' \in \mathsf{LWE}_s(f(m))$. To evaluate arbitrary functions, one must ensure that one bit of padding exists in the phase of the ciphertext, or utilize the so-called *full-domain* functional bootstrapping [CZB+22, LMP21, KS21] known in the literature.

2.3.2 Circuit Bootstrapping.

The circuit bootstrapping [CGGI17] was proposed to convert LWE ciphertext $\mathsf{LWE}_s(m)$ with $m \in \{0,1\}$ into a GGSW encryption of m, $\mathsf{GGSW}_{q,\mathbf{S}}^{B,\ell}(m)$. Despite the efficiency of the LHE mode using the GGSW encryption, the original circuit bootstrapping of [CGGI17] required repeated calls to the programmable bootstrapping and additional keyswitches, hindering the adoption of TFHE's LHE mode based on circuit bootstrapping. Later on, the circuit bootstrapping was greatly improved by [WWL+24] by employing many optimization techniques in the literature such as PBSManyLUT [CLOT21], HomTrace [CDKS21] and SchemeSwitch [DMKMS24], which was further improved in the work of [WHS+24].

3 Homomorphic Field Trace

In this section, we review the automorphism evaluation EvalAuto and homomorphic field trace in FHEW-like cryptosystmes. Then, we also review existing algorithms that tried to remove the phase amplification throughout two different domains.

3.1 Automorphisms and HomTrace

For a power-of-two N, we can define N different automorphisms $\tau_d : \mathcal{R} \to \mathcal{R}$ over $\mathcal{R}$ (or $\mathcal{R}_{q,N}$) for $d \in \mathbb{Z}_{2N}^{\times}$. Specifically, for an arbitrary polynomial $a(X) \in \mathcal{R}_{q,N}$, we have $\tau_d(a(X)) = a(X^d)$. This can be directly applied to GLWE-like ciphertexts (including

GLev, GGSW ciphertexts) to transform an encryption of $M(X)$ to $M(X^d)$. We denote the homomorphic automorphism evaluation algorithm $\mathsf{EvalAuto}$ in Algorithm 1. Note that the $\mathsf{EvalAuto}$ only adds keyswitching noise from line **2**, as the automorphism evaluation itself is a reordering of coefficients and does not add any noise. Thus, the variance of noise accumulated by $\mathsf{EvalAuto}$ can be bounded by the variance of GLWE keyswitch in Appendix B.2.

Algorithm 1: Homomorphic Automorphism $\mathsf{EvalAuto}(\mathbf{C}, d)$

Input: $\mathbf{C} = \mathsf{GLWE}_{q,\mathbf{S}}(M(X)) = (A_1, \cdots, A_k, B)$
Input: Automorphism index $d \in \mathbb{Z}_{2N}^{\times}$
Input: Switching key $\mathsf{Atk}_d = \mathsf{GLev}_{q,\mathbf{S}}^{B,\ell}(\tau_d(\mathbf{S}))$
Output: $\mathbf{C}' = \mathsf{GLWE}_{q,\mathbf{S}}(M(X^d))$
1 $\tilde{\mathbf{C}} \leftarrow (\tau_d(A_1), \cdots, \tau_d(A_k), \tau_d(B))$
2 $\mathbf{C}' \leftarrow \mathsf{GLWE.KS}\big(\tilde{\mathbf{C}}, \mathsf{Atk}_d\big)$
3 **return** $\mathbf{C}'$

Next, given a tower of finite fields $K = K_N \geq K_{N/2} \geq \ldots \geq K_1 = \mathbb{Q}$ where K_n denotes the $2n$-th cyclotomic field, the field trace $\mathrm{Tr}_{K/\mathbb{Q}} = \mathrm{Tr}_{K_2/K_1} \circ \ldots \circ \mathrm{Tr}_{K_N/K_{N/2}}$ can be evaluated with $\log N$ consecutive automorphisms. Specifically, in the work of [CDKS21], they utilize automorphisms with indices $d_{\mathsf{Tr},i} = 2^{\log N - i + 1} + 1$ for $i \in [\log N]$, and recursively perform $m_{i+1}(X) = m_i(X) + \tau_{d_{\mathsf{Tr},i}}(m_i(X))$ for $i = 1, \ldots, \log N$ for $m_1(X) \in \mathcal{R}_{q,N}$. To describe its effect on the polynomial, we first partition the indices in $\mathbb{Z}_N$ into $\log N + 1$ sets:

$$I_i = \{2^{i-1} + k \cdot 2^i \mod N \mid k \in \mathbb{N}\} \subset \mathbb{Z}_N$$

for $i \in [\log N + 1]$. Then, during i-th iteration, assume that $r \in [\log N + 1]$ and we observe the sign of the coefficients whose indices are in I_r:

$$\tau_{d_{\mathsf{Tr},i}}\left(X^{2^{r-1}+k\cdot 2^r}\right) = X^{2^{r-1}+k\cdot 2^r} \cdot X^{(2k+1)\cdot 2^{\log N + r - i}}.$$

If $r = i$, then $X^{(2k+1)\cdot 2^{\log N + r - i}} = X^{(2k+1)\cdot N} = -1 \mod (X^N + 1)$, and the sign is flipped after the automorphism evaluation. If $r > i$, then $X^{(2k+1)\cdot 2^{\log N + r - i}} = 1 \mod (X^N + 1)$, and the sign of these coefficients is preserved.

Then by calculating $m_{i+1}(X) = m_i(X) + \tau_{d_{\mathsf{Tr},i}}(m_i(X))$ during each i-th iteration, we can observe the coefficients whose indices correspond to the set I_i are 'zeroized', while the coefficients that correspond to the set I_j with $j > i$ are doubled. Thus after $\log N$ iterations, every coefficient except for the constant term is canceled out, while the constant coefficient is amplified by the factor $2^{\log N} = N$. Thus for a polynomial $\mu(X) = \mu_0 + \mu_1 X + \cdots \in \mathcal{R}_{q,N}$, we have

$$\mathrm{Tr}_{K/K_n}(\mu(X)) = (N/n) \cdot \sum_{i=0}^{n-1} \mu_{i\cdot N/n} \cdot X^{i\cdot N/n},$$
$$\mathrm{Tr}_{K/\mathbb{Q}}(\mu(X)) = N \cdot \mu_0,$$

for $n \mid N$. The detailed algorithm for $\mathsf{HomTrace}$ is depicted in Algorithm 2. While we denoted a general homomorphic trace of Tr_{K/K_n} for $n \mid 2N$ in Algorithm 2, we assume that $n = 1$ unless mentioned otherwise. That is, $\mathsf{HomTrace}(\mathbf{C})$ homomorphically evaluates the trace $\mathrm{Tr}_{K/\mathbb{Q}}(M(X)) = N \cdot M_0$.

Algorithm 2: Homomorphic Trace (Tr_{K/K_n}) evaluation, $\mathsf{HomTrace}(\mathbf{C}, n)$

Input: $\mathbf{C} = \mathsf{GLWE}_{q,\mathbf{S}}(M(X))$
Input: Power-of-two integer $n \leq N$
Input: Switching key $\mathsf{Atk}_d = \mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(\tau_d(\mathbf{S}))$, for $d \in \{2^{\log N - i + 1} + 1\}_{i \in [\log N]}$
Output: $\mathbf{C}' = \mathsf{GLWE}_{q,\mathbf{S}}\left(\mathrm{Tr}_{K/K_n}(M(X))\right)$

$\mathbf{C}' \leftarrow \mathbf{C}$
for $k = 1$ **to** $\log(N/n)$ **do**
    $\mathbf{C}' \leftarrow \mathbf{C}' + \mathsf{EvalAuto}(\mathbf{C}', 2^{\log N - k + 1} + 1)$
end
return $\mathbf{C}'$

3.2 Previous solutions for HomTrace phase amplification

3.2.1 NTT-Based Approach.

Ever since the HomTrace algorithm was proposed by [CDKS21], many works that employed the HomTrace [LSL+25, WWL+24] followed their approach and chose q, the ciphertext modulus coprime to N, so that the inverse of N existed in q. This approach was named the '**NTT-based**' approach in [WHS+24]. In this approach, as briefly mentioned in Section 2.3.2, the inverse of N, N^{-1} is first multiplied to the GLWE ciphertext and the HomTrace in Algorithm 2 is evaluated afterward. Then thanks to the N^{-1} multiplied, the phase amplification is naturally removed after the evaluation. We first analyze the noise growth for this method.

Theorem 1. *Let $\mathbf{C} \in \mathcal{R}^{k+1}_{q,N}$ be a GLWE ciphertext whose phase is $\mu(X) = \Delta \cdot M(X) + E(X)$ under the secret $\mathbf{S} = (S_1, \cdots, S_k)$, where the ciphertext modulus q is coprime to N. Also, assume the constant term of $M(X)$ is $M_0 \in \mathbb{Z}_q$. Then the* HomTrace *returns a ciphertext $\mathbf{C}' \in \mathsf{GLWE}_{q,\mathbf{S}}(M_0)$ with noise variance given as follows:*

$$\mathsf{Var}(\mathbf{C}') \leq \mathsf{Var}(\mathbf{C}) + \frac{N^2 - 1}{3} \cdot V_{\mathsf{Auto}}$$

where V_{Auto} denotes the noise variance of the EvalAuto *in Algorithm 1.*

Proof. See Appendix A.1 □

3.2.2 FFT-Based Approach.

While the mainstream of HomTrace was the NTT-Based approach due to the elimination of the factor N, the authors of [WHS+24] stress that under the large modulus of circuit bootstrapping, FFT tends to outperform NTT. This is illustrated in their results (Table 3 of [WHS+24]), where the FFT-based implementation shows a fivefold decrease in latency compared to the NTT-based implementation even with smaller parameters. Within the literature, they also provide a new preprocessing method based on modulus switching for FFT-based scenarios, where the existence of the inverse of N mod q is not guaranteed. While it is not mandatory, we assume $N \mid q$ for simplicity.

The main idea of preprocessing of [WHS+24] is to first switch the modulus of the (G)LWE ciphertext from $\mathcal{R}_{q,N}$ to $\mathcal{R}_{q/N,N}$ ($\mathbb{Z}_q^{n+1}$ to $\mathbb{Z}_{q/N}^{n+1}$, respectively) using the modulus switching in Appendix B.1. Then, they perform the *modulus raising* back to $\mathcal{R}_{q,N}$ ($\mathbb{Z}_q^{n+1}$, respectively), which interprets each coefficient in $\mathcal{R}_{q/N,N}$ as the same value in $\mathcal{R}_{q,N}$, i.e.,

for $M(X) \in \mathcal{R}_{t_1,N}$:

$$\mathsf{ModRaise}_{t_1 \to t_2}(M(X)) = M(X) \mod t_2,$$

for $t_1 \leq t_2$. During the modulus raising, it adds a polynomial $\frac{q}{N}U(X)$ for $U(X) \in \mathcal{R}$ upon the message, as the decryption is done in modulus q, not q/N. Luckily, this term $\frac{q}{N}U(X)$ gets removed by the scaling of factor N. We call their method PreHomTrace, which is presented in Algorithm 3.

Algorithm 3: PreHomTrace, from [WHS+24]

Input: $\mathbf{C} = \mathsf{GLWE}_{q,\mathbf{S}}(M(X))$, where $M(X) = M_0 + M_1X + \cdots$
Input: Automorphism keys $\mathsf{Atk}_d = \mathsf{GLev}^{B,\ell}_{q,\mathbf{S}'}(\tau_d(\mathbf{S}))$, for $d \in \{2^{\log N - i + 1} + 1\}_{i \in [\log N]}$
Output: $\bar{\mathbf{C}} = \mathsf{GLWE}_{q,\mathbf{S}}(M_0)$
$\mathbf{C}' \leftarrow \mathsf{ModRaise}_{q/N \to q} \circ \mathsf{ModSwitch}_{q \to q/N}(\mathbf{C})$
▷ $\mathbf{C}' \in \mathsf{GLWE}_{q,\mathbf{S}}\left(\frac{1}{N}M(X)\right) + \left(\mathbf{0}, \ldots, \mathbf{0}, \frac{q}{N}U(X)\right)$
$\bar{\mathbf{C}} \leftarrow \mathsf{HomTrace}(\mathbf{C}')$
return $\bar{\mathbf{C}}$

Theorem 2 (PreHomTrace From [WHS+24], Appendix G, Theorem 2)**.** *Let* $\mathbf{C} \in \mathcal{R}^{k+1}_{q,N}$ *be a GLWE ciphertext whose phase is* $\mu(X) = \Delta \cdot M(X) + E(X)$ *under the secret* $\mathbf{S} = (S_1, \cdots, S_k)$*. Also, assume the constant term of* $M(X)$ *is* $M_0 \in \mathbb{Z}_q$*. Then the* PreHomTrace *in Algorithm 3 returns a ciphertext* $\mathbf{C}' \in \mathsf{GLWE}_{q,\mathbf{S}}(M_0)$ *with noise variance given as follows:*

$$\mathsf{Var}(\mathbf{C}') \leq \mathsf{Var}(\mathbf{C}) + N^2 \cdot V_{\mathsf{MS}} + \frac{N^2 - 1}{3} \cdot V_{\mathsf{Auto}}$$

where V_{Auto} *denotes the noise variance of the* EvalAuto *in Algorithm 1, and* V_{MS} *denotes the noise variance from the modulus switching in Appendix B.1.*

Proof. See Appendix A.2. □

The results of Theorem 1 and Theorem 2 show that the additional error variance from the trace evaluation is $\mathcal{O}(N^3)$, as the V_{MS} and V_{Auto} introduce error with variance $\mathcal{O}(N)$ (see Appendix B.1 and B.2). While this cubic error can be hindered with well-chosen parameters or doesn't act as a tricky problem in low-precision mode, the authors of [WHS+24] emphasize that the error growth of $\mathcal{O}(N^3)$ is a huge obstacle for circuit bootstrapping to be used in high-precision operations.

Thus, the authors of [WHS+24] proposed a high precision HomTrace, dubbed **HP-HomTrace**, where they keyswitch the original GLWE ciphertext of dimension N and rank k into a larger rank GLWE ciphertext with rank $k' > k$. Then the large rank GLWE automorphism key can be generated with a much smaller noise variance. Thus, the absolute size of V_{Auto} decreases. However, we remark that the cubic error variance itself is not resolved even in this setting, and additional computational overhead is introduced due to performing HomTrace over large rank lowers the overall efficiency as well as the keyswitching to, and from larger rank GLWE. We present the HP-HomTrace algorithm in Algorithm 4 and analyze its noise variance in Theorem 3.

Algorithm 4: High Precision HomTrace, HP-HomTrace [WHS+24]

Input: $\mathbf{C} = \mathsf{GLWE}_{q,\mathbf{S}}(M(X))$, where $M(X) = M_0 + M_1 X + \cdots$
Input: GLWE KeySwitch key $\mathsf{KSK}_{\mathbf{S}\to\mathbf{S}'}$
Input: GLWE KeySwitch key $\mathsf{KSK}_{\mathbf{S}'\to\mathbf{S}}$
Input: Automorphism keys $\mathsf{Atk}_d = \mathsf{GLev}^{B,\ell}_{q,\mathbf{S}'}(\tau_d(\mathbf{S}))$, for $d \in \{2^{\log N - i + 1} + 1\}_{i \in [\log N]}$
Output: $\bar{\mathbf{C}} = \mathsf{GLWE}_{q,\mathbf{S}}(M_0)$

$\mathbf{C}' \leftarrow \mathsf{GLWE.KS}(\mathbf{C}, \mathsf{KSK}_{\mathbf{S}\to\mathbf{S}'})$
$\bar{\mathbf{C}} \leftarrow \mathsf{ModRaise}_{q/N\to q} \circ \mathsf{ModSwitch}_{q\to q/N}(\mathbf{C}')$
$\triangleright\ \bar{\mathbf{C}} \in \mathsf{GLWE}_{q,\mathbf{S}'}\left(\frac{1}{N}M(X)\right) + \left(\mathbf{0}, \ldots, \mathbf{0}, \frac{q}{N}U(X)\right)$
$\bar{\mathbf{C}} \leftarrow \mathsf{HomTrace}(\bar{\mathbf{C}})$
$\bar{\mathbf{C}} \leftarrow \mathsf{GLWE.KS}\left(\bar{\mathbf{C}}, \mathsf{KSK}_{\mathbf{S}'\to\mathbf{S}}\right)$
return $\bar{\mathbf{C}}$

Theorem 3 (HP-HomTrace From [WHS+24], Section 5.1, Theorem 4)**.** *Let $\mathbf{C} \in \mathcal{R}^{k+1}_{q,N}$ be a GLWE ciphertext whose phase is $\mu(X) = \Delta \cdot M(X) + E(X)$ under the secret $\mathbf{S} = (S_1, \cdots, S_k)$. Let $\mathbf{S}' = (S_1, \cdots, S'_k)$ be a GLWE secret key of rank k' such that $k' > k$. Moreover, assume that the constant term of $M(X)$ is $M_0 \in \mathbb{Z}_p$. Then the* HomTrace *evaluated using the HP-HomTrace of [WHS+24] returns a ciphertext $\mathbf{C}' \in \mathsf{GLWE}_{q,\mathbf{S}}(M_0)$ with noise variance given as follows:*

$$\mathsf{Var}(\mathbf{C}') \leq \mathsf{Var}(\mathbf{C}) + V_{\mathbf{S}\to\mathbf{S}'} + N^2 \cdot V_{\mathsf{MS},k'} + \frac{N^2 - 1}{3} \cdot V_{\mathsf{Auto},k'} + V_{\mathbf{S}'\to\mathbf{S}}$$

where $V_{\mathsf{Auto},k'}$ denotes the error variance of the EvalAuto *in Algorithm 1 under the large rank GLWE, and $V_{\mathsf{MS},k'}$ denotes the error variance from the modulus switching under the large rank GLWE.*

Proof. See Appendix A.3. □

4 RevHomTrace : Breaking the $\mathcal{O}(N^3)$ barrier

We now present our novel RevHomTrace algorithm that naturally removes the factor N, and only adds noise whose variance is bounded by $\mathcal{O}(N \log N)$. The algorithm is depicted in 5. This significantly improves the $\mathcal{O}(N^3)$ barrier of previously introduced homomorphic trace algorithms, only with a tradeoff of slight computational overhead from modulus switching.

The improvements of our RevHomTrace are based on two intuitions: **(1)** Rather than preprocessing GLWE ciphertext with modulus switching from q to q/N, we perform modulus switching from q to $q/2$ on each iteration before EvalAuto. **(2)** To ensure correctness, we evaluate the automorphisms in *reverse* order. These tweaks are depicted in Algorithm 5, and we discuss these in detail.

We analyze the behavior of ciphertext phase when evaluating two algorithms, PreHomTrace and RevHomTrace, on GLWE ciphertexts with $N = 8$. Figure 1 illustrates this comparison: Figure 1a shows the case for PreHomTrace, and Figure 1b shows the case for RevHomTrace.

In PreHomTrace, as discussed in Section 3.2.2, the preprocessing stage adds the polynomial $\frac{q}{N}U(X)$ upon the message exactly once. This polynomial exhibits distinct behavior depending on the index of the polynomial coefficient. For the constant coefficient, the

Algorithm 5: Amplification Removing Reverse Homomorphic Trace $\mathsf{RevHomTrace}(\mathbf{C}, n)$

Input: $\mathbf{C} = \mathsf{GLWE}_{q,\mathbf{S}}(M(X))$, where $M(X) = M_0 + M_1X + \cdots$
Input: Switching key $\mathsf{Atk}_d = \mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(\tau_d(\mathbf{S}))$, for $d \in \{2^i + 1\}_{i\in[\log N]}$
Output: $\mathbf{C}' = \mathsf{GLWE}_{q,\mathbf{S}}\left(\frac{n}{N} \cdot \mathrm{Tr}_{K/K_n}(M(X))\right)$

$\mathbf{C}' \leftarrow \mathbf{C}$
for $k = (\log n + 1)$ **to** $\log(N)$ **do**
  $\bar{\mathbf{C}} \leftarrow \mathsf{ModRaise}_{q/2\to q} \circ \mathsf{ModSwitch}_{q\to q/2}(\mathbf{C}')$
    ▷ $\bar{\mathbf{C}} = \mathbf{C}' + \left(0, \cdots, 0, \frac{q}{2}U_k(X) + E_{\mathsf{MS}}(X)\right)$
  $\mathbf{C}' \leftarrow \bar{\mathbf{C}} + \mathsf{EvalAuto}(\bar{\mathbf{C}}, 2^k + 1)$
end
return $\mathbf{C}'$

constant term of $\frac{q}{N}U(X)$ becomes amplified by a factor of N during subsequent iterations and naturally cancels out. For non-constant coefficients, during the $\mathsf{EvalAuto}$ stage, automorphisms negate these coefficients, causing both $U(X)$ and the messages to vanish.

Nonetheless, for $\mathsf{RevHomTrace}$, the modulus switching and raising operations ($q \to q/2 \to q$) add a polynomial $\frac{q}{2}U_i(X)$ at each i-th step. Then the coefficients that are fixed or negated by the automorphism are eliminated. For permuted coefficients, the coefficients of $U_i(X)$ and the message coefficients persist into $(i+1)$-th iteration. These persistence of $U_i(X)$ terms in permuted positions creates interference with other 'eliminated coefficients' in subsequent iterations if the automorphisms are evaluated in the same order as $\mathsf{PreHomTrace}$.

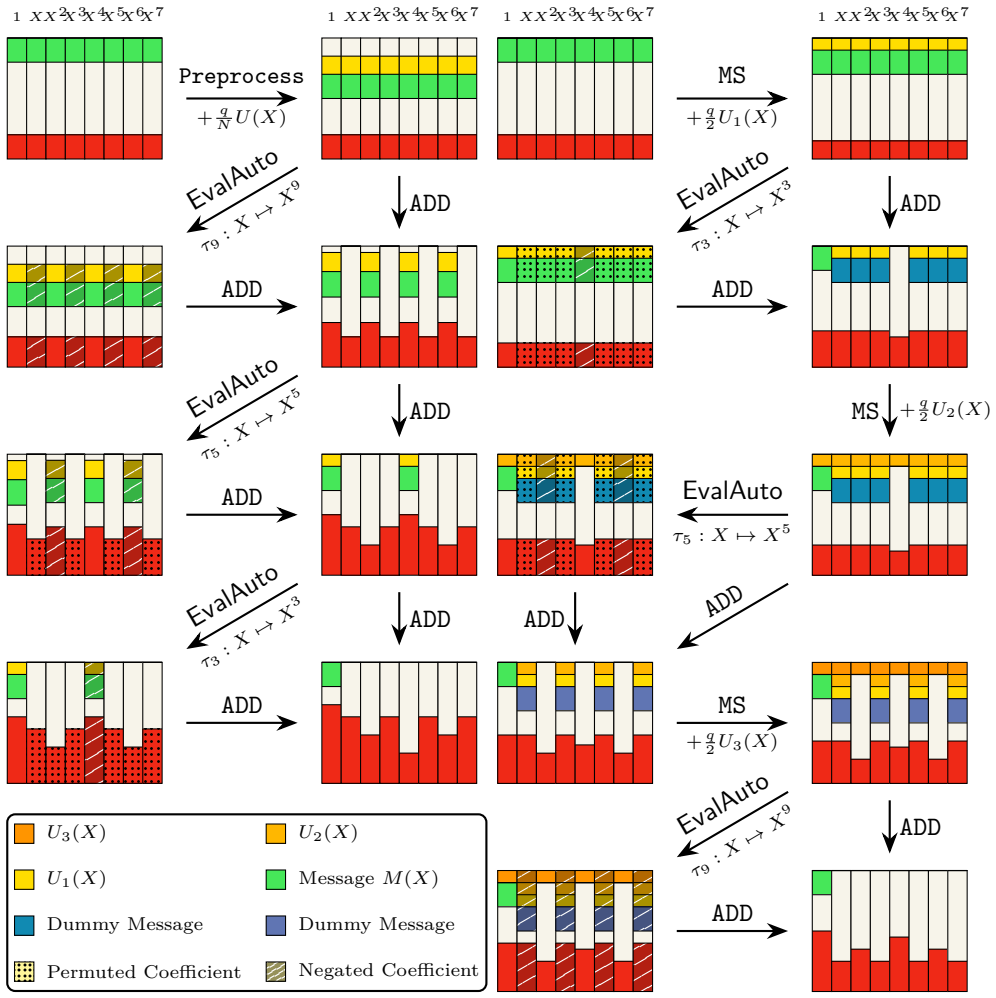
Thus we mitigated this issue by evaluating the automorphisms in reverse order. This ensures that coefficients eliminated through negation in previous iterations remain fixed in subsequent automorphism evaluations. For those coefficients, even if $\frac{q}{2}U_i(X)$ is added, it naturally cancels out as they are doubled. This is illustrated in Figure 1b, where the coefficient of X^4 is first eliminated by τ_3. Then, the coefficient is fixed in both τ_5 and τ_9, and the coefficient of $U_2(X)$ and $U_3(X)$ are naturally doubled and wiped out.

With these simple algorithmic tweak, our $\mathsf{RevHomTrace}$ algorithm still works correctly and even reduces the noise bound significantly. The analysis and the results are denoted in Theorem 4. Note that since the $\mathsf{ModSwitch}$ is actually a division by 2, and can be done in negligible time with shifting operation.Since $\mathsf{ModRaise}$ simply reinterprets the ciphertext from modulus $q/2$ to modulus q, it does not consume any additional time. Overall, $\mathsf{RevHomTrace}$ adds negligible computational overhead, the computational complexity remains almost the same as the original $\mathsf{HomTrace}$ and the preprocessed version of [WHS+24].

Theorem 4. *Let $\mathbf{C} \in \mathcal{R}^{k+1}_{q,N}$ be a GLWE ciphertext whose phase is $\mu(X) = \Delta \cdot M(X) + E(X)$ under the secret $\mathbf{S} = (S_1, \cdots, S_k)$. Also, assume the constant term of $M(X)$ is $M_0 \in \mathbb{Z}_q$. Then the $\mathsf{RevHomTrace}$ returns a ciphertext $\mathbf{C}' \in \mathsf{GLWE}_{q,\mathbf{S}}(M_0)$ with noise variance given as follows:*

$$\mathsf{Var}(\mathbf{C}') \le \mathsf{Var}(\mathbf{C}) + 4\log N \cdot V_{\mathsf{MS}} + \log N \cdot V_{\mathsf{Auto}}.$$

where V_{Auto} denotes the noise variance of the $\mathsf{EvalAuto}$ in Algorithm 1, and V_{MS} denotes the noise variance from the modulus switching.



(a) PreHomTrace(Alg. 3). (b) RevHomTrace (Alg. 5).

Figure 1: Comparison of the phase of the GLWE ciphertext with dimension $N = 8$ under two different homomorphic trace algorithms. Figure 1a depicts when PreHomTrace is applied, and Figure 1b depicts when RevHomTrace is applied. The `Preprocess` in Figure 1a refers to the modulus switching and raising in line 1 of Algorithm 3.

Proof. For the correctness of our algorithm, we adopt the index-set based approach. For $j \in [\log N]$, we examine the behavior of the ciphertext during the j-th loop of RevHomTrace. We partition the index sets into two different partitions:

$$P_{0,j} = \{I_1, \cdots, I_{\log N - j + 1}\}$$
$$P_{1,j} = \{I_{\log N - j + 2}, \cdots, I_{\log N + 1}\}$$

where $P_{0,j}$ represents the non-zeroized and non-preserved index sets during the previous iterations, while $P_{1,j}$ is the set of zeroized and well-preserved index sets. Note that for the constant set $I_{\log N+1} = \{0\}$, the constant term is not zeroized, but is always well-preserved for any automorphism. Thus we put $I_{\log N+1}$ in $P_{1,j}$ for the sake of correctness of our proof.

Correctness: At the start of the j-th iteration, assume the ciphertext encrypts a polynomial $\mu_j(X) = \mu_0 + \mu_1 X + \cdots \in \mathcal{R}_{q,N}$, where the coefficients whose indices are in the partition set $P_{1,j} \setminus \{I_{\log N+1}\}$ are all zero, and (generally) non-zero otherwise.

After the $\mathsf{ModRaise}_{q/2\to q} \circ \mathsf{ModSwitch}_{q\to q/2}$ operations, the ciphertext becomes an encryption of $\frac{1}{2}\mu_j(X) + \frac{q}{2}U_j(X)$. We then observe the behavior of

$$\frac{1}{2}\mu_j(X) + \frac{q}{2}U_j(X) + \frac{1}{2}\mu_j(X^{2^j+1}) + \frac{q}{2}U_j(X^{2^j+1})$$

where the addition is performed between the original ciphertext and the automorphism-evaluated one.

As mentioned in Section 3.1, the automorphism τ_{2^j+1} flips the sign of coefficients whose indices are in the set $I_{\log N-j+1}$, and preserves the coefficients in the index sets in partition $P_{1,j}$. Based on this property, we can calculate each coefficient of index $\ell \in \{0, 1, \cdots, N-1\}$:

For the phase part:

$$\frac{1}{2}\left(\mu_j(X) + \mu_j(X^{2^j+1})\right) = \begin{cases} \frac{1}{2}\cdot(\mu_{j,\ell} - \mu_{j,\ell}) = 0 & \text{for } \ell \in I_{\log N-j+1} \\ 0+0=0 & \text{for } \ell \in I_k \in P_{1,j} \setminus \{0\} \\ \frac{1}{2}(\mu_0+\mu_0) = \mu_0 & \text{for } \ell = 0 \\ \text{Some } \mu'_\ell \in \mathbb{Z}_q & \text{for } \ell \in I_k \in P_{0,j} \setminus \{I_{\log N-j+1}\}. \end{cases}$$

For the noise polynomial part:

$$\frac{q}{2}\left(U_j(X) + U_j(X^{2^j+1})\right) = \begin{cases} \frac{q}{2}\cdot(U_{j,\ell} - U_{j,\ell}) = 0 & & \text{for } \ell \in I_{\log N-j+1} \\ \frac{q}{2}\cdot(U_{j,\ell} + U_{j,\ell}) = 0 & (\mathrm{mod}\ q) & \text{for } \ell \in I_k \in P_{1,j} \\ \text{Some } U'_\ell \in \mathbb{B} & & \text{for } \ell \in I_k \in P_{0,j} \setminus \{I_{\log N-j+1}\}, \end{cases}$$

since $U_j(X)$ has binary coefficients $\{0,1\}$, the terms $\frac{q}{2}\cdot 2U_{j,\ell} = q\cdot U_{j,\ell} \equiv 0 \pmod q$ for preserved coefficients.

Combining these results, we observe that: **(1)** The coefficients with indices in $I_{\log N-j+1}$ get zeroized. Next, **(2)** The coefficients with indices in $P_{1,j}$ still remain zero after the j-th iteration, and due to the preservation of coefficients, they naturally cancel out the $\frac{q}{2}U_j(X)$ terms. Finally, **(3)** The constant term remains unchanged.

Thus, after the j-th iteration, we can update the partition as:

$$P_{0,j+1} = \{I_1, \cdots, I_{\log N-j}\}$$
$$P_{1,j+1} = \{I_{\log N-j+1}, \cdots, I_{\log N+1}\}$$

After the final $\log N$-th iteration, every set will be in $P_{1,\log N+1}$, meaning that every coefficient (except for the constant term) is well zeroized. As the constant term is left unchanged, the final output of $\mathsf{RevHomTrace}$ gives GLWE encryption of M_0.

Noise Analysis: we analyze how much noise is accumulated during our $\mathsf{RevHomTrace}$. Figure 2 depicts the error propagation during the j-th iteration of the $\mathsf{RevHomTrace}$.

Likewise in the proof of Theorem 1, we again build a recurrence relation on the error polynomial E_j after j-th iteration. We have:

$$E_j = \left(\frac{1}{2}E_{j-1} + E_{\mathsf{MS}}\right) + \tau_{2^j+1}\left(\frac{1}{2}\cdot E_{j-1} + E_{\mathsf{MS}}\right) + E_{\mathsf{Auto}},$$

where E_{MS} and E_{Auto} are the error increments by modulus switching and $\mathsf{EvalAuto}$, respectively. Representing this equation in terms of noise variance gives:

$$\mathsf{Var}(E_j) \le \mathsf{Var}(E_{j-1}) + 4V_{\mathsf{MS}} + V_{\mathsf{Auto}}.$$

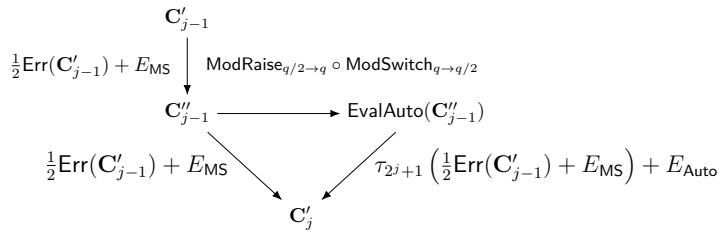


Figure 2: Error propagation in j-th iteration of RevHomTrace

Finally, solving this for $j = \log N$, and $E_0 = \mathsf{Var}(\mathbf{C})$, we get

$$\mathsf{Var}(\mathbf{C}') \le \mathsf{Var}(\mathbf{C}) + 4\log N \cdot V_{\mathsf{MS}} + \log N \cdot V_{\mathsf{Auto}}.$$

□

As we mentioned earlier, the noise variance of V_{Auto} is known to be $\mathcal{O}(N)$, and thus, our RevHomTrace achieves a noise propagation of variance $\mathcal{O}(N \log N)$, which is a huge improvement compared to the previous bound of $\mathcal{O}(N^3)$. Moreover, it can easily be employed in classic scenarios that use HomTrace with the removal of the factor N for free. For example, we can replace the preprocessing and the HomTrace in line 2 and 3 of Algorithm 4 into RevHomTrace to further reduce the noise from HomTrace.

5 Experimental Results

In this section, we provide experimental results based on our new RevHomTrace algorithm, and adapt them into previous applications (e.g., circuit bootstrapping [WHS+24,WWL+24], packing keyswitching [CDKS21]) and demonstrate the efficiency of our algorithm. We implemented our RevHomTrace on top of `TFHE-rs` [Zam22] and the FFT-based circuit bootstrapping implementation of [WHS+24][1]. They both use 64-bit unsigned integers for the ciphertext modulus $q = 2^{64}$. We ran our experiment on Ubuntu 24.04, running on Intel i9-13900k @ 5.80 GHz with 32 cores and 128 GB RAM. For reproducibility, our implementation is made available at https://github.com/Stirling75/RevHomTrace.

5.1 Performance of RevHomTrace

We first compare our RevHomTrace with the preprocessing version in Algorithm 3 (denoted as PreHomTrace), and the HP-HomTrace of Algorithm 4. For parameters for this setting, we referred to the parameter sets `INT_LHE_64` and `INT_LHE_256` from [WHS+24], which are represented in Table 2. As the keyswitching parameters for automorphism in their parameter sets are optimized for large rank GLWE $(A_1, A_2, B) \in \mathcal{R}_{q,N}^3$ with $(N, k') = (2048, 2)$, we optimized the parameters $(B_{\mathsf{Auto},\mathtt{k}},\ b_{\mathsf{Auto},k})$ for automorphism evaluation for $(N, k) = (2048, 1)$ based on the same decomposition length ℓ_{Auto}. This is due to the complexity of keyswitching being equal to $k \cdot (k+1) \cdot \ell$ polynomial multiplications (see Appendix B.2), and thereby leaving out the decomposition length affecting the efficiency.

For comparison, we generated GLWE encryptions of zero, i.e., $\mathsf{GLWE}_{q,\mathbf{S}}(0) \in \mathcal{R}_{q,N}^{k+1}$, and evaluated each algorithm over 1000 iterations. We measured the latency and the average absolute noise level, and the maximal noise level after the homomorphic trace evaluation. The results are as depicted in Table 3. From the results, we observe that the

[1] https://github.com/KAIST-CryptLab/refined-tfhe-lhe

Table 2: Two parameter sets from [WHS+24], where we added an optimized $B_{\mathsf{Auto},k}$ and $b_{\mathsf{Auto,k}}$ for reduced error EvalAuto in small GLWE rank k under the same $\ell_{Auto,k'}$. The σ represents the standard deviation of the error of GLWE ciphertext. The $b_{\mathsf{Auto}}, b_{\mathsf{k}\to\mathsf{k}'}, b_{\mathsf{k}'\to\mathsf{k}}$ represents the scaling bit of the *Split-FFT* that performs exact polynomial multiplication with negligible failure probability. For more details, check Section 3.2 of [WHS+24].

Param. Set	N	k	σ_k	k'	$\sigma_{k'}$	ℓ_{Auto}	$B_{\mathsf{Auto,k}}$	$b_{\mathsf{Auto,k}}$	$B_{\mathsf{Auto,k'}}$	$b_{\mathsf{Auto,k'}}$	$\ell_{k\to k'}$	$B_{k\to k'}$	$b_{k\to k'}$	$\ell_{k'\to k}$	$B_{k'\to k}$	$b_{k'\to k}$
$\texttt{INT\_LHE\_64}_{\mathsf{GLWE}}$	2048	1	$2^{14.06}$	2	2^2	4	2^{10}	2^{37}	2^{12}	2^{40}	3	2^{15}	2^{42}	3	2^{12}	2^{42}
$\texttt{INT\_LHE\_256}_{\mathsf{GLWE}}$	2048	1	$2^{14.06}$	2	2^2	6	2^7	2^{35}	2^9	2^{37}	3	2^{15}	2^{42}	4	2^{10}	2^{38}

Table 3: Comparison results of three different algorithms PreHomTrace, HP-HomTrace and RevHomTrace over two different parameter sets. The PreHomTrace represents the HomTrace evaluation after preprocessing in Algorithm 3. The Aver., and the Max. noise represents the average, and maximum absolute noise accumulated during the homomorphic trace evaluation for each algorithm expressed in bits.

Param. Set	Algorithm	Latency	Aver. Noise (bit)	Max. Noise (bit)
$\texttt{INT\_LHE\_64}_{\mathsf{GLWE}}$	PreHomTrace	795.7 μs	33.58	38.53
	HP-HomTrace	1.39 ms	31.61	33.72
	RevHomTrace	809.0 μs	**28.99**	**31.66**
$\texttt{INT\_LHE\_256}_{\mathsf{GLWE}}$	PreHomTrace	847.6 μs	31.03	36.00
	HP-HomTrace	1.74 ms	29.02	31.20
	RevHomTrace	867.8 μs	**26.49**	**29.18**

RevHomTrace achieves similar latency to the baseline algorithm PreHomTrace, but the average noise level is reduced by approximately 4 to 5 bits. Moreover, in terms of maximum noise level, our algorithm exhibits approximately 6 to 7 bits of noise reduction. Notably, our RevHomTrace surpasses the HP-HomTrace method in terms of both latency and noise level after homomorphic trace evaluation, as they suffer from additional key switchings to and from large rank GLWE ciphertext. From our parameters, additional key switchings of HP-HomTrace brought an approximately twofold increase in latency compared to both PreHomTrace and RevHomTrace.

We further examine the behavior of homomorphic trace functions in terms of ring dimension N. We leave other parameters fixed, including the noise standard deviation, and examine the noise level by changing the value of N. For comparison, we only use the parameter set $\texttt{INT\_LHE\_256}_{\mathsf{GLWE}}$. For HP-HomTrace, we naively fixed the setting $k = 1$ for small rank GLWE and $k' = 2$ for large rank. The results for $N = 2^8, \cdots, 2^{16}$ are represented in Figure 3. From the figure, the noise growth of PreHomTrace is considerable compared to the other two trace algorithms, where approximately one bit of noise growth occurs when the ring size is doubled. Also, at first glance, one might notice that the noise growth of HP-HomTrace is rather similar to our RevHomTrace, substantially different from the $\mathcal{O}(N^3)$ we mentioned earlier in Theorem 3. This is due to the extremely small noise standard deviation of $\sigma_{k'} = 2^2$, and the noise accumulation from $\mathsf{KeySwitch}_{\mathbf{S}\to\mathbf{S}'}$ and the HomTrace on larger rank GLWE is hindered by the keyswitching error from $\mathsf{KeySwitch}_{\mathbf{S}'\to\mathbf{S}}$, which accumulates noise with variance $\mathcal{O}(N)$. Still, our RevHomTrace outperforms these two algorithms regardless of the ring dimension, thanks to the $\mathcal{O}(N \log N)$ noise bound.

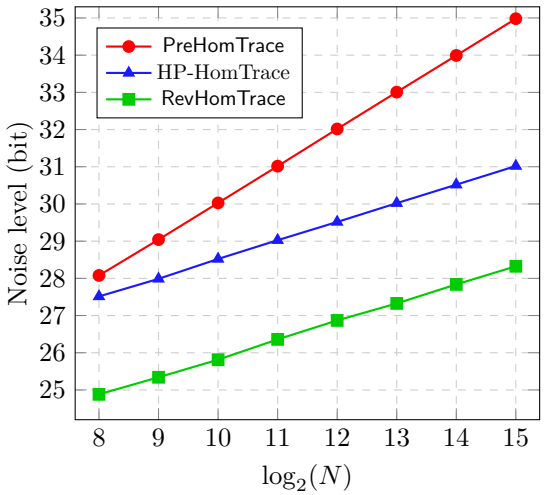


Figure 3: Comparison of average noise level of three different homomorphic trace evaluation methods, PreHomTrace, HP-HomTrace and RevHomTrace over different ring dimension. The x-axis represents the ring dimension $\log_2 N$, and the y-axis represents the bit level of accumulation noise.

5.2 Improved High Precision Circuit Bootstrapping

As the state-of-the-art circuit bootstrapping methods highly rely on homomorphic trace evaluation for canceling out the non-constant terms from intermediate GLev ciphertext, our RevHomTrace can be utilized to reduce the overall noise. While our RevHomTrace also improves the noise analysis in the bitwise mode of circuit bootstrapping, which permits large noise variance, we found that we can dramatically improve the conversion step of the high precision circuit bootstrapping mode of [WHS+24], which they call it *integer input LHE* mode. We denote this mode as HP-LHE mode for simplicity. We briefly introduce the HP-LHE mode of circuit bootstrapping and present the results by replacing the preprocessing and the HP-HomTrace step with our RevHomTrace.

1. **Refr. & Extr.** : $\mathsf{LWE}_s(q/2^{\delta-i\cdot\tau}\cdot\sum_{j=i\cdot\tau}^{\delta-1} m_j\cdot 2^j)\to\left\{\mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(m'_j+\cdots)\right\}_{j\in[i\cdot\tau,i\cdot\tau+(\tau-1)]}$

   Refresh and extract τ bits from the least significant bit (LSB) of the encrypted plaintext with PBSManyLUT [CLOT21] without sample extraction. Here, $m'_j = m_{(i+1)\cdot\tau-1}\oplus m_j$ for $j\in[i\cdot\tau, i\cdot\tau+(\tau-2)]$, and $m_j = m_{i\cdot\tau+(\tau-1)}$ for $j = i\cdot\tau+(\tau-1)$.

2. **Conv.** : $\mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(m'_j+\cdots)\to\mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(m'_j)\to\mathsf{GGSW}^{B,\ell}_{q,\mathbf{S}}(m'_j)$ for $j\in[0,\tau-1]$

   Conversion of the GLev ciphertext into GGSW ciphertext by three consecutive steps:

   Preprocess : $\mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(m'_j+\cdots)\longrightarrow\mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}\left(\frac{1}{N}\cdot m'_j+\cdots\right)$

   HP−HomTrace : $\mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}\left(\frac{1}{N}\cdot m'_j+\cdots\right)\longrightarrow\mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(m'_j)$

   Evaluate the high precision homomorphic trace function, HomTrace on the preprocessed GLev ciphertext.

   SchemeSwitch : $\mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(m'_j)\longrightarrow\mathsf{GGSW}^{B,\ell}_{q,\mathbf{S}}(m'_j)$

   Evaluate the scheme switching that converts GLev ciphertext into GGSW ciphertext that encrypts same message m.

Table 4: Comparison between the HP-LHE circuit bootstrapping of [WHS+24] with HP-HomTrace, and replaced version with our RevHomTrace. The maximal depth indicates the CMux depth with failure probability less than 2^{-40}.

Param. Set	Method	δ	τ	Latency (ms) Refr.	Conv.	Total		Maximal Depth (F.P $< 2^{-40}$)
INT_LHE_64	[WHS+24]	6	1	148.24	35.80	184.04		1016
	RevHomTrace	6	1	146.45	22.24	**168.69**	(**1.09**×)	1082
	RevHom. + $\text{Tweak}_{4,9}$	6	1	189.21	22.41	211.62	(0.86×)	**9466**
	[WHS+24]	6	2	77.08	35.63	112.71		897
	RevHomTrace	6	2	74.31	22.25	**96.56**	(**1.17**×)	955
	RevHom. + $\text{Tweak}_{4,9}$	6	2	94.70	22.32	117.02	(0.96×)	**8439**
	[WHS+24]	6	3	50.97	34.99	85.96		140
	RevHomTrace	6	3	50.25	22.86	**73.11**	(**1.18**×)	149
	RevHom. + $\text{Tweak}_{4,9}$	6	3	63.16	23.15	86.31	(1.00×)	**1450**
INT_LHE_256	[WHS+24]	8	1	279.94	122.79	402.73		12478
	RevHomTrace	8	1	274.70	63.48	**338.18**	(**1.19**×)	12961
	RevHom. + $\text{Tweak}_{5,8}$	8	1	327.14	62.85	389.99	(1.03×)	**48441**
	[WHS+24]	8	2	139.88	123.72	263.6		7107
	RevHomTrace	8	2	139.59	63.47	**203.06**	(**1.30**×)	7418
	RevHom. + $\text{Tweak}_{5,8}$	8	2	169.32	66.03	235.35	(1.12×)	**27725**

3. **Remove.** : $\mathsf{LWE}_s(q/2^{\delta-i\cdot\tau}\cdot\sum_{j=i\cdot\tau}^{\delta-1} m_j\cdot 2^j) \to \mathsf{LWE}_s(q/2^{\delta-(i+1)\cdot\tau}\cdot\sum_{j=(i+1)\cdot\tau}^{\delta-1} m_j\cdot 2^j)$

   Remove the τ-bits from the LSB of encrypted plaintext, and repeat the procedure until all plaintext bits are removed.

Summing up, each iteration of the HP-LHE requires a single PBSManyLUT during the first step, whose complexity is similar to a single programmable bootstrapping. Also,τ times of the conversion from GLev to GGSW ciphertext (which requires ℓ_{CBS} times of HP-HomTrace and a single SchemeSwitch evaluation). Since we repeat this iteration δ/τ times, the total complexity becomes δ/τ-Programmable bootstrapping (PBS) in Step 1, and $\delta\ell$-$\mathsf{GLWE.KS}_{k\to k'}$, $\delta\ell \log N$-$\mathsf{GLWE.KS}_{k'\to k'}$, $\delta\ell$-$\mathsf{GLWE.KS}_{k'\to k}$ and $\delta\ell$-$\mathsf{GLWE.KS}_{k\to k}$ in Step 2.

We evaluated high-precision circuit bootstrapping with INT_LHE_64 and INT_LHE_256 of [WHS+24], using our optimized $(B_{\mathsf{Auto},k}, \ell_{\mathsf{Auto},k})$ from Table 2 and the extra parameters in Table 7. As summarized in Table 4, the latency of conversion step was nearly halved, bringing a speedup of at least 1.09× and up to 1.30×. Moreover, the noise after was also reduced, thereby increasing maximal circuit depth after bootstrapping. Removing HP-HomTrace further reduced key size and eliminated switching keys, making HP-LHE more efficient.

From our results of improvement, we also wanted to observe how much CMux depth of the HP-LHE we can achieve while maintaining similar latency from that of [WHS+24]. We found out that due to the reduced HomTrace error, the largest portion of error comes from the programmable bootstrapping step, PBSManyLUT. Thus, we adjusted the decomposition level and the base of the programmable bootstrapping, from $(\ell_{\mathsf{PBS}}, B_{\mathsf{PBS}}) = (3, 2^{11})$ to $(4, 2^9)$, and from $(4, 2^9)$ to $(5, 2^8)$, for INT_LHE_64 and INT_LHE_256, respectively. Note that increasing the decomposition length brings growth in latency by a factor of $\ell_{\mathsf{After}}/\ell_{\mathsf{Before}}$,

Algorithm 6: Homomorphic Packing algorithm of LWE ciphertexts `MS-PackLWEs`

Input: GLWE-Converted LWE ciphertexts $\mathbf{C}_j = \mathsf{GLWE}_{q,\mathbf{S}}(m_j + \cdots)$ for $j \in [2^\ell]$
Input: Automorphism key $\mathsf{Atk}_d = \mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(\tau_d(\mathbf{S}))$, for $d \in \{2^k+1\}_{k\in[\log N]}$

```
if ℓ = 0 then
    return C ← C_0
end
else
    C_Odd ← MS-PackLWEs({C_{2j-1}}_{j∈[2^{ℓ-1}]})
    C_Even ← MS-PackLWEs({C_{2j}}_{j∈[2^{ℓ-1}]})
    C̃ ← C_Even − X^{N/2^ℓ} · C_Odd
    C̃ ← ModRaise_{q/2→q} ∘ ModSwitch_{q→q/2}(C̃)
    C ← C̃ + EvalAuto(C̃, 2^ℓ + 1)
    C ← C + X^{N/2^ℓ} · C_Odd
end
return C
```

where ℓ_{After} denotes the modified decomposition length, and ℓ_{Before} denotes the original decomposition length. The benchmark results for these 'tweaked' parameters are also presented in Table 4, denoted as RevHom.+Tweak. With these tweaks, we observe that without severe performance degradation, we can achieve approximately 4 to 10 times of computational depth increment.

5.3 Application to PackLWEs

Next, we present a new `PackLWEs` algorithm that we call `MS-PackLWEs` that packs $n = 2^\ell$ LWE ciphertexts in a GLWE ciphertext with reduced error. The original algorithm comes from [CDKS21], and is still used in many works based on homomorphic encryption [KDE+23, MW24, ADDG24, HYT+24, BZS+24]. We briefly explain the core idea of their algorithm. We start from $n = 2^\ell$ LWE ciphertexts $c_j = (a_{j,0}, \cdots, a_{j,kN-1}, b_j) \in \mathsf{LWE}_{\overline{\mathbf{S}},q}(m_j)$ each encrypting $m_j \in \mathbb{Z}_q$ for $j \in [n]$. Note that $\overline{\mathbf{S}} \in \mathbb{Z}^{k\cdot N}$ is an LWE representation of the GLWE secret key $\mathbf{S} \in \mathcal{R}^k_{q,N}$. Then the reordering of the coefficients into polynomials gives a GLWE encryption of $m_j + \cdots \in \mathcal{R}_{q,N}$ under the key $\mathbf{S}$, $\mathbf{C}_{1,j} = \mathsf{GLWE}_{q,\mathbf{S}}(m_j + \cdots)$. The goal of the `PackLWEs` is to generate a GLWE encryption of $\sum_{i=1}^n m_i X^{(i-1)\cdot N/n}$. The work of [CDKS21] achieves this goal by two separate steps. First, they aggregate n GLWE ciphertexts using a $\log n = \ell$-layer binary tree. For $k \in [\ell]$, the aggregation is done by a single EvalAuto:

$$\mathbf{C}'_{k+1,j} = \left(\mathbf{C}_{k,j} + X^{N/2^k}\mathbf{C}_{k,j+N/2^k}\right) + \mathsf{EvalAuto}\left(\mathbf{C}_{k,j} - X^{N/2^k}\mathbf{C}_{k,j+N/2^k}, 2^k+1\right)$$

for $j \in [n/2^k]$. Thus, during each level, the $n/2^{k-1}$ ciphertexts are aggregated with $n/2^k$-EvalAuto. After ℓ level, the final ciphertext $\mathbf{C}'$ encrypts $M_\ell(X) = n \cdot \sum_{i=1}^n m_i X^{(i-1)\cdot N/n}$, but also encrypts non-zeroized dummy values. Next, the second step is held by evaluating the homomorphic partial trace $\mathsf{HomTrace}(\mathbf{C}', n)$ to further zeroize non-zeroized coefficients. We refer the reader to [CDKS21] for more details.

We present three variants of the `PackLWEs` algorithm. The first applies modulus switching to q/N on input LWE ciphertexts and revert them back to q, following the original approach. The second, `HP-PackLWEs`, switches GLWE ciphertexts under $\mathbf{S} \in \mathcal{R}^k_{q,N}$ to higher rank $k' > k$, evaluates `PackLWEs`, then switches back to rank k. The third, `MS-PackLWEs`, adapts RevHomTrace by modifying the aggregation step of [CDKS21] to eliminate the

Table 5: Comparison between five packing algorithms that pack multiple LWE ciphertexts in GLWE ciphertext. n is the number of input LWE ciphertexts, and N denotes the GLWE ring dimension. Also, D_{LWE} denotes the dimension of the LWE ciphertext, and ℓ_{Auto}, $\ell_{\mathsf{k}'}$, ℓ_{KS} denote the length of the automorphism key, rank switching key to rank k' and packing keyswitching key for the method of [CGGI17], respectively.

Algorithm	Complexity	Noise Accumulation	Key Size
`PackLWEs`	$\mathcal{O}\left(k^2 N \ell_{\mathsf{Auto}} \log N \cdot \max\{n, \log(N/n)\}\right)$	$\mathcal{O}\left(N^3\right)$	$\mathcal{O}(kN \log q)$
`HP-PackLWEs`	$\mathcal{O}(kk' N \log N \cdot \max\{n\ell_{k'}, (n + \log(N/n))\ell_{\mathsf{Auto}}\})$	$\mathcal{O}\left(N^3\right)$	$\mathcal{O}(k' N \log q)$
`MS-PackLWEs`	$\mathcal{O}\left(k^2 N \ell_{\mathsf{Auto}} \log N \cdot \max\{n, \log(N/n)\}\right)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(kN \log q)$
`PackKS`	$\mathcal{O}(k\ell_{\mathsf{KS}} D_{\mathsf{LWE}} \cdot N \log N)$	$\mathcal{O}(nD_{\mathsf{LWE}})$	$\mathcal{O}(kND_{\mathsf{LWE}} \log q)$
`PackKS-rs`	$\mathcal{O}(k\ell_{\mathsf{KS}} D_{\mathsf{LWE}} \cdot nN)$	$\mathcal{O}(nD_{\mathsf{LWE}})$	$\mathcal{O}(kND_{\mathsf{LWE}} \log q)$

noise polynomial $\frac{q}{2}U(X)$ from modulus switching by:

$$\mathbf{C}'_{k+1,j} = \frac{1}{2}\left(\mathbf{C}_{k,j} - X^{N/2^k}\mathbf{C}_{k,j+N/2^k}\right) + \mathsf{EvalAuto}\left(\frac{1}{2}\left(\mathbf{C}_{k,j} - X^{N/2^k}\mathbf{C}_{k,j+N/2^k}\right), 2^k+1\right) + X^{N/2^k}\mathbf{C}_{k,j+N/2^k}.$$

We depict our `MS-PackLWEs` in Algorithm 6, and due to excessive space, we present the rest of the algorithms and proofs throughout Appendices D.1, D.2 and D.3. Also, the complexity for each algorithm is presented in Table 5.

For comparison, we ran our experiment on three different `PackLWEs` algorithms we introduced, with the parameter sets $\texttt{INT\_LHE\_64}_{\mathsf{GLWE}}$ and $\texttt{INT\_LHE\_256}_{\mathsf{GLWE}}$ in Section 5.1. Also, for further comparison, we compared our result with the traditional packing keyswitching from [CGGI17, MS18], where [MS18] is a binary decomposition version of [CGGI17]. We refer the reader to Alg. 1 of [CGGI17], or Alg. 3 of [MS18] for further details. We abbreviate their method as `PackKS`. Moreover, the `TFHE-rs` library [Zam22] provides [CGGI17]-style packing[2] , and it is more efficient when the number of LWE ciphertexts are small. Their method first converts each LWE ciphertext into its own GLWE ciphertext, then rotates and sums them. We abbreviate the `TFHE-rs` method as `PackKS-rs`.

The results are presented in Table 6. From the results, `PackLWEs` shows the lowest latency among the five algorithms in all cases followed by our `MS-PackLWEs`. In contrast, `HP-PackLWEs` incurs nearly twice the latency of both `PackLWEs` and `MS-PackLWEs`. Also, the automorphism-based packing methods outperforms traditional packing methods in all cases.

For noise level, the traditional packing methods `PackKS` and `PackKS-rs` are better than automorphism-based methods when the number of LWE ciphertexts n is small. Nonetheless, as shown in Table 6, as their noise accumulation is proportional to n, their noise level get higher than our `MS-PackLWEs` if n grows. Also, we can observe our `MS-PackLWEs` outperforms automorphism-based packing methods `PackLWEs` and `HP-PackLWEs` in terms of noise level. Compared to the original `PackLWEs`, our `MS-PackLWEs` reduces about 8 to 9 bits of error from the ciphertext with about 10% increase of latency.

[2] https://github.com/zama-ai/tfhe-rs/.../algorithms/lwe_packing_keyswitch.rs

Table 6: Comparison between three PackLWEs algorithms. The Aver. Noise represents average absolute noise in bits. The Max. Noise represents Maximum absolute noise in bits. The PackLWEs indicates the PackLWEs algorithms from [CDKS21] with modulus switching, and the HP-PackLWEs and MS-PackLWEs indicate the PackLWEs algorithm based on the idea of HP-HomTrace of [WHS+24], and our RevHomTrace, respectively. The PackKS denote the packing key switching in [CGGI17, MS18], and PackKS-rs denote the TFHE-rs [Zam22] implementation of the packing keyswitching.

n	Method	INT_LHE_64$_{\mathsf{GLWE}}$			INT_LHE_256$_{\mathsf{GLWE}}$		
		Latency (ms)	Aver Noise (bit)	Max. Noise (bit)	Latency (ms)	Aver. Noise (bit)	Max. Noise (bit)
4	PackLWEs	**0.89**	32.81	39.49	**0.95**	30.21	36.87
	HP-PackLWEs	1.80	31.64	33.72	2.32	29.04	31.22
	MS-PackLWEs	0.99	29.01	31.72	1.05	26.33	29.10
	PackKS	1987	**28.82**	31.07	3331	**26.17**	**28.32**
	PackKS-rs	21.49	28.83	**31.01**	33.75	26.18	28.37
32	PackLWEs	**3.37**	33.05	40.16	**3.73**	30.43	37.55
	HP-PackLWEs	7.57	31.61	33.72	9.10	29.02	31.20
	MS-PackLWEs	3.74	**29.02**	**31.85**	4.11	**26.34**	**29.27**
	PackKS	1998	30.32	32.53	3367	27.68	29.86
	PackKS-rs	162.79	30.33	32.52	277.64	27.68	29.88
256	PackLWEs	**25.99**	35.60	40.51	**27.50**	33.00	37.92
	HP-PackLWEs	59.08	31.62	33.74	70.74	29.02	31.22
	MS-PackLWEs	27.99	**29.05**	**32.17**	30.09	**26.55**	**29.62**
	PackKS	1974	31.83	34.00	3330	29.19	31.37
	PackKS-rs	1312	31.83	34.01	2234	29.19	31.38
2048	PackLWEs	**263.68**	38.60	40.78	**278.34**	35.99	38.16
	HP-PackLWEs	581.43	31.66	33.79	659.79	29.03	31.21
	MS-PackLWEs	281.73	**30.28**	**32.45**	294.93	**27.66**	**29.84**
	PackKS	1999	33.32	35.52	3403	30.68	32.86
	PackKS-rs	10810	33.32	35.50	17689	30.69	32.81

6 Conclusion

In this paper, we propose an efficient homomorphic trace algorithm RevHomTrace. Our modulus-switching based approach removed the phase amplification problem, while breaking the traditional noise accumulation bound of HomTrace from $\mathcal{O}(N^3)$ to $\mathcal{O}(N \log N)$. We believe our improvement towards the homomorphic trace evaluation can help utilize the HomTrace and its applied algorithms in various fields of homomorphic encryption, including circuit bootstrapping, scheme switching, and many more.

Acknowledgement

This work was supported by the IITP(Institute of Information & Coummunications Technology Planning Evaluation)-ITRC(Information Technology Research Center) grant funded by the Korea government(Ministry of Science and ICT)(IITP-2025-RS-2022-00164800)

A Proofs

A.1 Proof of Theorem 1

Proof. During the preprocessing step, the multiplication by $N^{-1} \pmod q$ modifies the phase of the ciphertext into $N^{-1} \cdot \mu(X) = N^{-1} \cdot \Delta \cdot M(X) + N^{-1} \cdot E(X)$. After i-th iteration, the phase of the ciphertext becomes $\mathrm{Tr}_{K/K_{N/2^i}}(N^{-1} \cdot \mu(X)) + E_{\mathsf{Auto},i}$, where the $E_{\mathsf{Auto},i}$ represents the error induced from the $\mathsf{EvalAuto}$ after the i-th iteration. Thus, after $\log N$ iterations, we get $\mathrm{Tr}_{K/\mathbb{Q}}(N^{-1} \cdot \mu(X)) + E_{\mathsf{Auto},\log N}$, where $\mathrm{Tr}_{K/\mathbb{Q}}(N^{-1} \cdot \mu(X)) = N \cdot (N^{-1} \cdot \Delta \cdot M_0 + N^{-1} \cdot E_0) = \Delta \cdot M_0 + E_0$.

We now move on to the noise estimation of $\mathsf{HomTrace}$. Figure 4 depicts the error propagation during the i-th iteration of the $\mathsf{HomTrace}$.

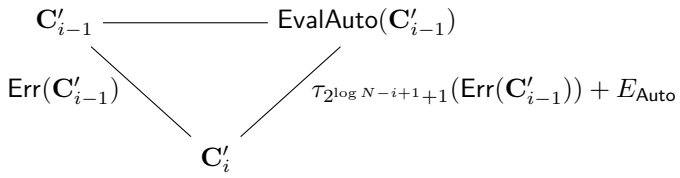


Figure 4: Error propagation in i-th iteration of **NTT-based** $\mathsf{HomTrace}$

Then we can build a recurrence relation on the error polynomial E_i, the error polynomial after i-th iteration. We have:

$$E_i = E_{i-1} + \tau_{2^{\log N-i+1}+1}(E_{i-1}) + E_{\mathsf{Auto}},$$

where E_{Auto} is the error increment by $\mathsf{EvalAuto}$. Solving this equation gives:

$$\mathsf{Var}(E_i) \le 4 \cdot \mathsf{Var}(E_{i-1}) + V_{\mathsf{Auto}},$$

and solving this recurrence relation for $i = \log N$ gives

$$\mathsf{Var}(\mathbf{C}') \le \mathsf{Var}(\mathbf{C}) + \sum_{i=0}^{\log N-1} 4^i V_{\mathsf{auto}} \le \mathsf{Var}(\mathbf{C}) + \frac{N^2-1}{3} V_{\mathsf{Auto.}}$$

□

A.2 Proof of Theorem 2

Proof. After the preprocessing, we start off from a GLWE encryption $\mathbf{C}'' = \mathsf{GLWE}_{q,\mathbf{S}}\left(\frac{\Delta}{N} \cdot M + \frac{q}{N} U\right)$ where $U \in \mathcal{R}$. The overall proof is the same as in the proof of Theorem 1, except the initial noise differs from $N^{-1} \cdot E$ to $\frac{1}{N} E + E_{\mathsf{MS}}$, where additional modulus switching error is added. Thus, after the $\mathsf{HomTrace}$ evaluation, we attain the GLWE ciphertext $\mathbf{C}'$ whose phase is

$$\begin{aligned} &\mathrm{Tr}_{K/\mathbb{Q}}\left(\frac{\Delta}{N} \cdot M\right) + \mathrm{Tr}_{K/\mathbb{Q}}\left(\frac{q}{N} U\right) + E_{\mathsf{init}} + N \cdot E_{\mathsf{MS}} + E_{\mathsf{Tr}} \\ &= \Delta \cdot M_0 + E_{\mathsf{init}} + N \cdot E_{\mathsf{MS}} + E_{\mathsf{Tr}}. \end{aligned}$$

Thus, the final noise variance becomes

$$\mathsf{Var}(\mathbf{C}') \leq \mathsf{Var}(\mathbf{C}) + N^2 \cdot V_{\mathsf{MS}} + \frac{N^2-1}{3} \cdot V_{\mathsf{Auto}}$$

□

A.3 Proof of Theorem 3

Proof. The kind of noise inside the output ciphertext is quite similar to that of Theorem 2, except for the additional keyswitching to, and from the large rank GLWE. Also, due to the larger rank, $V_{\mathsf{MS},k'}$ and $V_{\mathsf{Auto},k'}$ also differ from that of Theorem 2. These additional noises are accumulated in the ciphertext, thereby inducing a final variance of the HP-HomTrace function as

$$\mathsf{Var}(\mathbf{C}') \leq \mathsf{Var}(\mathbf{C}) + V_{\mathbf{S}\to\mathbf{S}'} + N^2 \cdot V_{\mathsf{MS},k'} + \frac{N^2-1}{3} \cdot V_{\mathsf{Auto},\mathsf{k}'} + V_{\mathbf{S}'\to\mathbf{S}},$$

where $N^2 \cdot V_{\mathsf{MS},k'}$ term corresponds to the V_{pre} term, and $\frac{N^2-1}{3} \cdot V_{\mathsf{Auto},\mathsf{k}'}$ corresponds to V_{tr} of Theorem 4 of [WHS+24].

□

B (G)LWE operations

Here, we review the algorithms and their corresponding noise amplification that we did not handle in the paper. Typically, we review the ModSwitch and the GLWE Keyswitching algorithm, which is an underlying core algorithm of our RevHomTrace.

B.1 Modulus Switching

Suppose we are given two different ciphertext moduli q and q' with $q' < q$. Suppose we are given a GLWE ciphertext $\mathbf{C} = (A_1, \cdots, A_k, B) \in \mathsf{GLWE}_{q,\mathbf{S}}(M)$ under the secret key $\mathbf{S} = (S_1, \cdots, S_k)$ with scaling factor Δ. Then the modulus switching from q to q', i.e., $\mathsf{ModSwitch}_{q\to q'}(\mathbf{C})$ outputs a GLWE ciphertext $\mathbf{C}' = (A'_1, \cdots, A'_k, B') \in \mathsf{GLWE}_{q',\mathbf{S}}(M)$ with scaling factor $\left\lfloor \frac{q'}{q} \right\rceil \Delta$, where

$$A'_i = \left\lfloor \frac{q'}{q} A_i \right\rceil \quad \text{for } i \in [k],$$
$$B' = \left\lfloor \frac{q'}{q} B \right\rceil.$$

Lemma 1 (Modulus Switching)**.** *Given a GLWE ciphertext $\mathbf{C} = (A_1, \cdots, A_k, B) \in \mathcal{R}_{q,N}^{k+1}$ whose phase is $\mu \in \mathcal{R}_{q,N}$ under the key $\mathbf{S} = (S_1, \cdots, S_k)$, the $\mathsf{ModSwitch}_{q\to q'}(\mathbf{C})$ for $q' < q$ outputs a GLWE ciphertext $\mathbf{C}' \in (A'_1, \cdots, A'_k, B') \in \mathcal{R}_{q'}^{k+1}$ under the key $\mathbf{S}$, whose phase is $\frac{q'}{q}\mu + E_{\mathsf{MS}}$. Then, the additional error E_{MS} has a variance of*

$$V_{\mathsf{MS}} \leq \frac{1+kN}{12},$$

if $\mathbf{S}$ *is a binary key.*

Proof. From the aforementioned process of modulus switching, we have

$$A'_i = \left\lfloor \frac{q'}{q} A_i \right\rceil \quad \text{for } i \in [k],$$
$$B' = \left\lfloor \frac{q'}{q} B \right\rceil.$$

Thus, following the approach of [BJSW25, WHS+24], we can write each term as

$$A'_i = \frac{q'}{q} A_i + E_i \quad \text{for } i \in [k],$$
$$B' = \frac{q'}{q} B + E_{k+1},$$

where each E_i's are polynomials whose coefficients are uniformly sampled from $\left[-\frac{1}{2}, \frac{1}{2}\right)$. Then, the phase of $\mathbf{C}'$ can be evaluated as:

$$\begin{aligned} B' - \sum_{i=1}^{k} A'_i \cdot S_i &= \left(\frac{q'}{q} B + E_{k+1}\right) - \left(\frac{q'}{q} \sum_{i=1}^{k} A_i \cdot S_i + \sum_{i=1}^{k} E_i \cdot S_i\right) \\ &= \frac{q'}{q}\left(B - \sum_{i=1}^{k} A_i \cdot S_i\right) + \left(E_{k+1} - \sum_{i=1}^{k} E_i \cdot S_i\right). \end{aligned}$$

Then, we get $E_{\mathsf{MS}} = E_{k+1} - \sum_{i=1}^{k} E_i \cdot S_i$, and we can estimate its variance by:

$$\begin{aligned} \mathsf{Var}_{\mathsf{MS}} &= \mathsf{Var}\left(E_{k+1} - \sum_{i=1}^{k} E_i \cdot S_i\right) \\ &\leq \mathsf{Var}(E_{k+1}) + \mathsf{Var}(\sum_{i=1}^{k} E_i \cdot S_i) \\ &\leq \frac{1}{12} + k \cdot N \cdot \frac{1}{12} = \frac{1 + kN}{12}, \end{aligned}$$

assuming S_i is binary.

□

B.2 GLWE Keyswitch

Suppose we want to switch the secret key of GLWE ciphertext from $\mathbf{S} = (S_1, \cdots, S_k) \in \mathcal{R}_{q,N}^k$ to $\mathbf{S}' \in \mathcal{R}_{q,N}^{k'}$. Then, using the GLWE switching key $\mathsf{KSK}_{\mathbf{S}\to\mathbf{S}'} = \left\{\mathsf{GLev}_{q,\mathbf{S}'}^{B_{\mathsf{KS}},\ell_{\mathsf{KS}}}(S_i)\right\}_{i\in[k]}$, we can perform the GLWE-to-GLWE key switching using Algorithm 7. The B_{KS} and ℓ_{KS} indicate the decomposition base and the length of the GLWE switching key, respectively. From the k iterations of gadget product that requires $(k'+1)\cdot\ell$ polynomial multiplications, the complexity for a single keyswitching becomes $k \cdot (k'+1) \cdot \ell$ polynomial multiplications over $\mathcal{R}_{q,N}$.

Lemma 2 (GLWE Keyswitching)**.** *Given a GLWE ciphertext* $\mathbf{C} = (A_1, \cdots, A_k, B) \in \mathcal{R}_{q,N}^{k+1}$

Algorithm 7: GLWE KeySwitching, $\mathsf{GLWE.KS}$

Input: $\mathbf{C} \in \mathsf{GLWE}_{q,\mathbf{S}}(M)$, where $\mathbf{S} = (S_1, \cdots, S_k) \in \mathcal{R}_{q,N}^k$
Input: KeySwitching key $\mathsf{KSK}_{\mathbf{S}\to\mathbf{S}'} = \left\{\mathsf{GLev}_{q,\mathbf{S}'}^{B_{\mathsf{KS}},\ell_{\mathsf{KS}}}(S_i)\right\}_{i\in[k]}$
Output: $\mathbf{C}' \in \mathsf{GLWE}_{q,\mathbf{S}'}(M)$
1 $\mathbf{C} = (A_1, \cdots, A_k, B) \in \mathcal{R}_{q,N}^{k+1}$
2 $\mathbf{C}' = (\mathbf{0}, B) \in \mathcal{R}_{q,N}^{k'+1}$
3 **for** $i = 1$ **to** k **do**
4 $\quad \mathbf{C}' \leftarrow \mathbf{C}' - A_i \odot \mathsf{GLev}_{q,\mathbf{S}'}^{B_{\mathsf{KS}},\ell_{\mathsf{KS}}}(S_i)$
5 **end**

whose phase is $\mu \in \mathcal{R}_{q,N}$ under the key $\mathbf{S} = (S_1, \cdots, S_k) \in \mathcal{R}_{q,N}^k$. Then the GLWE keyswitching in Algorithm 7 returns a GLWE ciphertext whose phase is $\mu + E_{\mathsf{KS}}$ under the key $\mathbf{S}' = (S'_1, \cdots, S'_{k'}) \in \mathcal{R}_{q,N}^{k'}$, where the variance V_{KS} of E_{KS} is given as:

$$V_{\mathsf{KS}} \leq k \cdot N \cdot \left(\frac{q^2 - B_{\mathsf{KS}}^{2\ell_{\mathsf{KS}}}}{24 B_{\mathsf{KS}}^{2\ell_{\mathsf{KS}}}} + \frac{1}{16}\right) + k \cdot \ell_{\mathsf{KS}} \cdot N \cdot V_{\mathsf{KSK}} \cdot \left(\frac{B_{\mathsf{KS}}^2 + 2}{12}\right),$$

where V_{KSK} is the noise variance of the keyswitching key, KSK.

Proof. We follow the footsteps of [CLOT21] for the proof of noise analysis of GLWE Keyswitch. The main error of the GLWE keyswitch is accumulated from the gadget product $\odot$ of line 4, thus we analyze the behavior from the gadget product. From the gadget decomposition, we gadget-decompose each A_i's in terms of

$$\tilde{A}_i = \sum_{j=1}^{\ell_{\mathsf{KS}}} \tilde{A_{i,j}} \cdot \left\lceil \frac{q}{B_{\mathsf{KS}}^j} \right\rfloor,$$

where $A'_{i,j}$ has uniform coefficients in $[\![-B_{\mathsf{KS}}/2, B_{\mathsf{KS}}/2[\![$. Then, we can write $\tilde{A}_i = A_i + \overline{A_i}$, where the coefficients of $\overline{A_i}$ are uniform in $[\![-q/2B_{\mathsf{KS}}^{\ell_{\mathsf{KS}}}, q/2B_{\mathsf{KS}}^{\ell_{\mathsf{KS}}}[\![$. Then, the analysis from [CLOT21] shows that the variance and the mean of each polynomial is given as $\mathsf{Var}(\tilde{A_{i,j,k}}) = \frac{B_{\mathsf{KS}}^2 - 1}{12}$, $\mathsf{Var}(\overline{A_{i,j}}) = \frac{q^2}{12 B_{\mathsf{KS}}^{2\ell}} - \frac{1}{12}$, and $E(\tilde{A_{i,j,k}}) = E(\overline{A_{i,j}}) = -\frac{1}{2}$. Thus, from the decryption of $\mathbf{C}' = (A'_1, \cdots, A_{k'}, B')$ by $\mathbf{S}'$, we get

$$\begin{aligned}
B' - \sum_{i=1}^{k'} A'_i \cdot S_i &= B - \sum_{i=1}^{k} \sum_{j=1}^{\ell_{\mathsf{KS}}} \tilde{A_{i,j}} \left(\left\lceil \frac{q}{B_{\mathsf{KS}}^j} \right\rfloor \cdot S_i + E_{i,j} \right) \\
&= B - \sum_{i=1}^{k} \left((A_i + \overline{A_i}) \cdot S_i + \sum_{j=1}^{\ell} \tilde{A_{i,j}} \cdot E_{i,j} \right) \\
&= \mu - \sum_{i=1}^{k} \overline{A_i} \cdot S_i - \sum_{i=1}^{k} \sum_{j=1}^{\ell} \tilde{A_{i,j}} \cdot E_{i,j}
\end{aligned}$$

where the coefficients of $E_{i,j}$ have variance $\mathsf{Var}(E_{i,j,k}) = V_{\mathsf{KSK}}$. Finally, our E_{KS} is represented as

$$E_{\mathsf{KS}} = \sum_{i=1}^{k} \overline{A_i} \cdot S_i + \sum_{i=1}^{k} \sum_{j=1}^{\ell} \tilde{A_{i,j}} \cdot E_{i,j}.$$

Using the result from [CLOT21], we can represent the variance as

$$\begin{aligned} V_{\mathsf{KS}} &\leq k \cdot N \cdot \left\{ \left(\frac{q^2}{12 B_{\mathsf{KS}}^{2\ell_{\mathsf{KS}}}} - \frac{1}{12} \right) \cdot \left(\frac{1}{4} \cdot \frac{1}{4} \right) + \frac{1}{4} \cdot \frac{1}{4} \right\} \\ &\qquad + k \cdot \ell_{\mathsf{KS}} \cdot N \cdot V_{\mathsf{KSK}} \left\{ \frac{B_{\mathsf{KS}^2} - 1}{12} + \frac{1}{4} \right\} \\ &\leq k \cdot N \cdot \left(\frac{q^2 - B_{\mathsf{KS}}^{2\ell_{\mathsf{KS}}}}{24 B_{\mathsf{KS}}^{2\ell_{\mathsf{KS}}}} + \frac{1}{16} \right) + k \cdot \ell_{\mathsf{KS}} \cdot N \cdot V_{\mathsf{KSK}} \cdot \left(\frac{B_{\mathsf{KS}}^2 + 2}{12} \right). \end{aligned}$$

□

C Parameters

Here, we present additional parameters we used for the experiment of the HP-LHE mode from Section 5.2. As the GLWE parameters are the same as those from Table 2, we present the parameters for LWE, PBS, and CBS.

Table 7: The parameter sets `INT_LHE_64` and `INT_LHE_256` from [WHS+24], and our tweaked version $\texttt{INT\_LHE\_64} + \text{Tweak}_{4,9}$ and $\texttt{INT\_LHE\_256} + \text{Tweak}_{5,8}$ used for HP-LHE experiment in Section 5.2.

Param. Set	n	σ_{LWE}	$\ell_{\mathsf{LWE.KS}}$	$B_{\mathsf{LWE.KS}}$	ℓ_{PBS}	B_{PBS}	ℓ_{CBS}	B_{CBS}
`INT_LHE_64`	873	$2^{44.55}$	2	7	3	2^{11}	4	2^5
$\texttt{INT\_LHE\_64} + \text{Tweak}_{4,9}$					4	2^9		
`INT_LHE_256`	953	$2^{42.55}$	2	7	4	2^9	8	2^3
$\texttt{INT\_LHE\_256} + \text{Tweak}_{5,8}$					5	8		

D Packing LWEs to RLWE

D.1 `PackLWEs`

The `PackLWEs` algorithm of [CDKS21] is given in Algorithm 8. After `PackLWEs`, $\log(N/2^\ell)$ times $\mathsf{EvalAuto}$ are executed to zeroize the coefficients that do not contain any messages.

Lemma 3. *Let* $\left\{ c_i = \mathsf{LWE}_{\overline{\mathbf{S}}}(m_i) \right\}_{i \in [n]}$ *be the set of* $n = 2^\ell$ *LWE ciphertexts under the LWE-representation of the GLWE key* $\mathbf{S}$ *with error variance* V_{in}*. Then the* `PackLWEs` *after the preprocessing, followed by zeroization, i.e., homomorphic evaluation of* Tr_{K/K_n} *returns a GLWE ciphertext of* $\mathbf{C}' = \mathsf{GLWE}_{q,\mathbf{S}} \left(\sum_{i=1}^{n} m_i X^{(i-1) \cdot N/n} \right)$ *whose error variance is given as follows:*

$$\mathsf{Var}(\mathbf{C}') \leq V_{\mathsf{in}} + N^2 \cdot V_{\mathsf{MS}} + \frac{N^2 n^2 - 1}{3n^2} \cdot V_{\mathsf{Auto}}.$$

Proof. After preprocessing by N, we have (G)LWE ciphertexts whose variance is $V_{\mathsf{input}} \leq \frac{1}{N^2} V_{\mathsf{in}} + V_{\mathsf{MS}}$. Next, from the proof of [CDKS21], we have the recurrence relation

$$E_{i+1} \leq 2E_i + E_{\mathsf{Auto}}$$

Algorithm 8: LWEs-to-GLWE, `PackLWEs` from [CDKS21]

Input: GLWE-Converted LWE ciphertexts $\mathbf{C}_j = \mathsf{GLWE}_{q,\mathbf{S}}(m_j + \cdots)$ for $j \in [2^\ell]$
Input: Automorphism key $\mathsf{Atk}_d = \mathsf{GLev}^{B,\ell}_{q,\mathbf{S}}(\tau_d(\mathbf{S}))$, for $d \in \{2^k+1\}_{k\in[\log N]}$

```
if ℓ = 0 then
    return C ← C_0
end
else
    C_Odd ← PackLWEs({C_{2j-1}}_{j∈[2^{ℓ-1}]})
    C_Even ← PackLWEs({C_{2j}}_{j∈[2^{ℓ-1}]})
    C ← (C_Even + X^{N/2^ℓ} · C_Odd) + EvalAuto(C_Even − X^{N/2^ℓ} · C_Odd, 2^ℓ + 1)
end
return C
```

for $i \in [\ell]$, during the evaluation of the aggregation tree. Thus, following the proof in Appendix A.1, we get an intermediate GLWE ciphertext whose error variance in valid coefficients is bounded by:

$$V_{\mathsf{inter}} \le n^2 \cdot V_{\mathsf{input}} + \frac{n^2-1}{3} V_{\mathsf{KS}}.$$

Then after the zeroization, i.e., homomorphic evaluation of Tr_{K/K_n}, we get a GLWE ciphertext $\mathbf{C}' = \mathsf{GLWE}_{q,\mathbf{S}}\left(\sum_{i=1}^{n} m_i X^{(i-1)\cdot N/n}\right)$ with

$$\begin{aligned}\mathsf{Var}(\mathbf{C}') &\le (N/n)^2 V_{\mathsf{inter}} + \frac{(N/n)^2-1}{3} V_{\mathsf{Auto}} \\ &\le V_{\mathsf{in}} + N^2 \cdot V_{\mathsf{MS}} + \frac{N^2n^2-1}{3n^2} \cdot V_{\mathsf{Auto}}.\end{aligned}$$

□

D.2 `HP-PackLWEs`

Despite the work of [WHS+24] did not present a high precision LWEs-to-GLWE algorithm; we inherited their idea of HP-HomTrace and built a high precision `PackLWEs` algorithm, which we call `HP-PackLWEs`. One thing to notice is that the GLWE-converted ciphertexts are first key-switched into a larger rank GLWE under the key $\mathbf{S}'$, which induces an additional 2^ℓ calls to the GLWE keyswitching algorithm. Also, it is first preprocessed by N.

Lemma 4. *Let $\left\{c_i = \mathsf{LWE}_{\overline{\mathbf{S}}}(m_i)\right\}_{i\in[n]}$ be the set of $n = 2^\ell$ LWE ciphertexts under the LWE-representation of the GLWE key $\mathbf{S}$ with error variance V_{in}. Then the* `HP-PackLWEs` *in Algorithm 9 returns a GLWE ciphertext of $\mathbf{C}' = \mathsf{GLWE}_{q,\mathbf{S}}\left(\sum_{i=1}^{n} m_i X^{(i-1)\cdot N/n}\right)$ whose error variance is given as follows:*

$$\mathsf{Var}(\mathbf{C}') \le V_{\mathsf{in}} + N^2 \cdot V_{\mathbf{S}\to\mathbf{S}'} + N^2 \cdot V_{\mathsf{MS}} + \frac{N^2n^2-1}{3n^2} \cdot V_{\mathsf{Auto},k'} + V_{\mathbf{S}'\to\mathbf{S}},$$

where

Proof. The proof directly follows from the proof of Lemma 3, and the only difference is the keyswitching from and to the large rank, and that the EvalAuto is held on large rank

Algorithm 9: High precision LWEs-to-GLWE, `HP-PackLWEs`

Input: GLWE-Converted LWE ciphertexts $\mathbf{C}_j = \mathsf{GLWE}_{q,\mathbf{S}}(m_j + \cdots)$ for $j \in [2^\ell]$
Input: Keyswitching Key $\mathsf{KSK}_{\mathbf{S}\to\mathbf{S}'}$
Input: Keyswitching Key $\mathsf{KSK}_{\mathbf{S}'\to\mathbf{S}}$
Input: Automorphism key $\mathsf{Atk}_d = \mathsf{GLev}^{B,\ell}_{q,\mathbf{S}'}(\tau_d(\mathbf{S}))$, for $d \in \{2^k+1\}_{k\in[\log N]}$

for $j = 1$ **to** n **do**
  $\mathbf{C}'_j \leftarrow \mathsf{GLWE.KS}(\mathbf{C}_j, \mathsf{KSK}_{\mathbf{S}\to\mathbf{S}'})$
  $\mathbf{C}'_j \leftarrow \mathsf{ModRaise}_{q/N\to q} \circ \mathsf{ModSwitch}_{q\to q/N}(\mathbf{C}')$
end
$\mathbf{C}' \leftarrow \mathtt{PackLWEs}\left(\{\mathbf{C}'_j\}_{j\in[n]}, \mathsf{Atk}\right)$
for $i = \log n$ **to** $\log N - 1$ **do**
  $\mathbf{C}' \leftarrow \mathbf{C}' + \mathsf{EvalAuto}(\mathbf{C}', 2^{i+1}+1)$
end
$\mathbf{C} \leftarrow \mathsf{GLWE.KS}(\mathbf{C}', \mathsf{KSK}_{\mathbf{S}'\to\mathbf{S}})$
return $\mathbf{C}$

GLWE. Thus, the input noise variance is changed into $V_{\mathsf{input}} \le \frac{1}{N^2} V_{\mathsf{in}} + V_{\mathsf{MS}} + V_{\mathbf{S}\to\mathbf{S}'}$. After the `HP-PackLWEs` and the zeroization, the error variance becomes:

$$\mathsf{Var}(\mathbf{C}'') \le V_{\mathsf{in}} + N^2 \cdot V_{\mathbf{S}\to\mathbf{S}'} + N^2 \cdot V_{\mathsf{MS}} + \frac{N^2n^2-1}{3n^2} \cdot V_{\mathsf{Auto},k'},$$

where $V_{\mathsf{KS},k'}$ is the keyswitching error variance over rank k' GLWE ciphertext. Finally, the rank k' GLWE ciphertext is keyswitched into rank k, accumulating an additional keyswitch error:

$$\mathsf{Var}(\mathbf{C}'') \le V_{\mathsf{in}} + N^2 \cdot V_{\mathbf{S}\to\mathbf{S}'} + N^2 \cdot V_{\mathsf{MS}} + \frac{N^2n^2-1}{3n^2} \cdot V_{\mathsf{Auto},k'} + V_{\mathbf{S}'\to\mathbf{S}}.$$

□

D.3 `MS-PackLWEs`

Lemma 5. *Let* $\left\{c_i = \mathsf{LWE}_{\overline{\mathbf{S}}}(m_i)\right\}_{i\in[n]}$ *be the set of* $n = 2^\ell$ *LWE ciphertexts under the LWE-representation of the GLWE key* $\mathbf{S}$ *with error variance* V_{in}*. Then the* `MS-PackLWEs` *in Algorithm 6 returns a GLWE ciphertext of* $\mathbf{C}' = \mathsf{GLWE}_{q,\mathbf{S}}\left(\sum_{i=1}^n m_i X^{(i-1)\cdot N/n}\right)$ *whose error variance is given as follows:*

$$\mathsf{Var}(\mathbf{C}') \le V_{\mathsf{in}} + 4\log N \cdot V_{\mathsf{MS}} + \log N \cdot V_{\mathsf{Auto}}$$

Proof. We observe the behavior of the aggregation during level-ν for $\nu \in [\ell]$. Without loss of generality, we assume we aggregate two ciphertexts $\mathbf{C}^\nu_{\mathsf{Even}}$ and $\mathbf{C}^\nu_{\mathsf{Odd}}$. For each phase of these two ciphertexts, we only observe the coefficients located at every $N/2^\nu$-th position:

$$\mu^\nu_{\mathsf{E}}(X) = \sum_{i=0}^{2^{\nu-1}-1} \left(M^\nu_{\mathsf{E},i} + e^\nu_{\mathsf{E},i}\right) \cdot X^{i\cdot N/2^{\nu-1}} + X^{N/2^\nu} \cdot \sum_{i=0}^{2^{\nu-1}-1} E^\nu_{\mathsf{E},i} \cdot X^{i\cdot N/2^{\nu-1}},$$

$$\mu^\nu_{\mathsf{O}}(X) = \sum_{i=0}^{2^{\nu-1}-1} \left(M^\nu_{\mathsf{O},i} + e^\nu_{\mathsf{O},i}\right) \cdot X^{i\cdot N/2^{\nu-1}} + X^{N/2^\nu} \cdot \sum_{i=0}^{2^{\nu-1}-1} E^\nu_{\mathsf{O},i} \cdot X^{i\cdot N/2^{\nu-1}}.$$

In short, the phase contains the message and some small error in every $N/2^{\nu-1}$-th position, and contains dummy values $(E_{\mathsf{E}}, E_{\mathsf{O}})$) in other coefficients. Then after the aggregation

step:

$$\mathbf{C}_{\nu+1} = \frac{1}{2}\left(\mathbf{C}^{\nu}_{\mathsf{Even}} - X^{N/2^{\nu}}\mathbf{C}^{\nu}_{\mathsf{Odd}}\right) + \mathsf{EvalAuto}\left(\frac{1}{2}\left(\mathbf{C}^{\nu}_{\mathsf{Even}} - X^{N/2^{\nu}}\mathbf{C}^{\nu}_{\mathsf{Odd}}\right), 2^{\nu}+1\right) + X^{N/2^{\nu}}\mathbf{C}^{\nu}_{\mathsf{Odd}},$$

the phase of $\mathbf{C}_{\nu+1}$ is equal to

$$\mu_{\nu+1}(X) = \frac{1}{2}\left(\mu^{\nu}_{\mathsf{E}}(X) + \mu^{\nu}_{\mathsf{E}}(X^{2^{\nu}+1}) - X^{\frac{N}{2^{\nu}}}\cdot\mu^{\nu}_{\mathsf{O}}(X) + X^{\frac{N}{2^{\nu}}}\cdot\mu^{\nu}_{\mathsf{O}}(X^{2^{\nu}+1})\right) + \frac{q}{2}U_{\nu}(X) + \frac{q}{2}U_{\nu}(X^{2^{\nu}+1}) + e_{\mathsf{MS}}(X) + e_{\mathsf{MS}}(X^{2^{\nu}+1}) + e_{\mathsf{Auto}}(X) + X^{\frac{N}{2^{\nu}}}\cdot\mu^{\nu}_{\mathsf{O}}(X).$$

Then again, for the coefficients located at every $N/2^{\nu}$-th position:

$$\mu_{\nu+1,k} = \begin{cases} M^{\nu}_{\mathsf{E},j} + e^{\nu}_{\mathsf{E},j} + 2e_{\mathsf{MS},k} + e_{\mathsf{Auto},k} & \text{for } k = \frac{N}{2^{\nu-1}}\cdot j \\ M^{\nu}_{\mathsf{O},j} + e^{\nu}_{\mathsf{O},j} + e_{\mathsf{Auto},k} & \text{for } k = \frac{N}{2^{\nu-1}}\cdot j + \frac{N}{2^{\nu}} \end{cases}$$

for $j \in [0, 2^{\nu-1}-1]$. Then again, we get the recurrence relation from the proof of Theorem 4:

$$V_{k+1} \leq V_k + 4V_{\mathsf{MS}} + V_{\mathsf{Auto}}.$$

Thus, we conclude that

$$\begin{aligned}\mathsf{Var}(\mathbf{C}') &\leq V_{\mathsf{input}} + (4\log n \cdot V_{\mathsf{MS}} + \log n \cdot V_{\mathsf{Auto}}) + (4\log(N/n)\cdot V_{\mathsf{MS}} + \log(N/n)\cdot V_{\mathsf{Auto}}) \\ &\leq V_{\mathsf{in}} + 4\log N \cdot V_{\mathsf{MS}} + \log N \cdot V_{\mathsf{Auto}}.\end{aligned}$$

□

References

[ADDG24] Martin R Albrecht, Alex Davidson, Amit Deo, and Daniel Gardham. Crypto dark matter on the torus: Oblivious prfs from shallow prfs and tfhe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 447–476. Springer, 2024.

[BGV14] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.

[BJSW25] Olivier Bernard, Marc Joye, Nigel P Smart, and Michael Walter. Drifting towards better error probabilities in fully homomorphic encryption schemes. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 181–211. Springer, 2025.

[Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In *Annual cryptology conference*, pages 868–886. Springer, 2012.

[BZS+24] Song Bian, Zian Zhao, Ruiyu Shen, Zhou Zhang, Ran Mao, Dawei Li, Yizhong Liu, Masaki Waga, Kohei Suenaga, Zhenyu Guan, et al. Chloe: Loop transformation over fully homomorphic encryption via multi-level vectorization and control-path reduction. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 35–35. IEEE Computer Society, 2024.

[CDKS21] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient homomorphic conversion between (ring) lwe ciphertexts. In *International conference on applied cryptography and network security*, pages 460–479. Springer, 2021.

[CGGI17] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster packed homomorphic operations and efficient circuit bootstrapping for tfhe. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 377–408. Springer, 2017.

[CGGI20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I 23*, pages 409–437. Springer, 2017.

[CLOT21] Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for tfhe. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 670–699. Springer, 2021.

[CZB+22] Pierre-Emmanuel Clet, Martin Zuber, Aymen Boudguiga, Renaud Sirdey, and Cédric Gouy-Pailler. Putting up the swiss army knife of homomorphic calculations by means of tfhe functional bootstrapping. *Cryptology ePrint Archive*, 2022.

[DM15] Léo Ducas and Daniele Micciancio. Fhew: bootstrapping homomorphic encryption in less than a second. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 617–640. Springer, 2015.

[DMKMS24] Gabrielle De Micheli, Duhyeong Kim, Daniele Micciancio, and Adam Suhl. Faster amortized fhew bootstrapping using ring automorphisms. In *IACR International Conference on Public-Key Cryptography*, pages 322–353. Springer, 2024.

[FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, 2012.

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009.

[GHPS13] Craig Gentry, Shai Halevi, Chris Peikert, and Nigel P Smart. Field switching in bgv-style homomorphic encryption. *Journal of Computer Security*, 21(5):663–684, 2013.

[GHS12] Craig Gentry, Shai Halevi, and Nigel P Smart. Better bootstrapping in fully homomorphic encryption. In *International Workshop on Public Key Cryptography*, pages 1–16. Springer, 2012.

[HYT+24] Jiaxing He, Kang Yang, Guofeng Tang, Zhangjie Huang, Li Lin, Changzheng Wei, Ying Yan, and Wei Wang. Rhombus: Fast homomorphic matrix-vector multiplication for secure two-party inference. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 2490–2504, 2024.

[KDE+23] Andrey Kim, Maxim Deryabin, Jieun Eom, Rakyong Choi, Yongwoo Lee, Whan Ghang, and Donghoon Yoo. General bootstrapping approach for rlwe-based homomorphic encryption. *IEEE Transactions on Computers*, 73(1):86–96, 2023.

[KLD+23] Andrey Kim, Yongwoo Lee, Maxim Deryabin, Jieun Eom, and Rakyong Choi. Lfhe: fully homomorphic encryption with bootstrapping key size less than a megabyte. *Cryptology ePrint Archive*, 2023.

[KS21] Kamil Kluczniak and Leonard Schild. Fdfb: Full domain functional bootstrapping towards practical fully homomorphic encryption. *arXiv preprint arXiv:2109.02731*, 2021.

[LMP21] Zeyu Liu, Daniele Micciancio, and Yuriy Polyakov. Large-precision homomorphic sign evaluation using fhew/tfhe bootstrapping. *Cryptology ePrint Archive*, 2021.

[LPR10] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29*, pages 1–23. Springer, 2010.

[LSL+25] Zhihao Li, Xuan Shen, Xianhui Lu, Ruida Wang, Yuan Zhao, Zhiwei Wang, and Benqiang Wei. Leveled functional bootstrapping via external product tree. *Cryptology ePrint Archive*, 2025.

[MS18] Daniele Micciancio and Jessica Sorrell. Ring packing and amortized fhew bootstrapping. *Cryptology ePrint Archive*, 2018.

[MW24] Samir Jordan Menon and David J Wu. {YPIR}:{High-Throughput}{Single-Server}{PIR} with silent preprocessing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5985–6002, 2024.

[Reg09] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.

[WHS+24] Ruida Wang, Jincheol Ha, Xuan Shen, Xianhui Lu, Chunling Chen, Kunpeng Wang, and Jooyoung Lee. Fhew-like leveled homomorphic evaluation: Refined workflow and polished building blocks. *Cryptology ePrint Archive*, 2024.

[WWL+24] Ruida Wang, Yundi Wen, Zhihao Li, Xianhui Lu, Benqiang Wei, Kun Liu, and Kunpeng Wang. Circuit bootstrapping: faster and smaller. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 342–372. Springer, 2024.

[Zam22] Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. https://github.com/zama-ai/tfhe-rs.

[Zam25] Zama. TFHE-rs: A (Practical) Handbook. https://github.com/zama-ai/tfhe-rs-handbook, 2025.